

ANALISIS KOMPARATIF ARSITEKTUR FUSI STEREO UNTUK ESTIMASI POSE 6D DARI VIDEO STEREOSKOPIK

SKRIPSI

Sebagai Salah Satu Syarat untuk Meraih
Gelar Sarjana Komputer

Oleh:
Moreno Kristovan
32210013



Fakultas Teknologi Dan Desain
Program Studi Informatika
Universitas Bunda Mulia
Tangerang
2025

**UNIVERSITAS BUNDA MULIA
FAKULTAS TEKNOLOGI DAN DESAIN
PROGRAM STUDI INFORMATIKA**

Persetujuan Skripsi

Yang bertanda tangan dibawah ini, menyatakan bahwa Skripsi dengan judul

**ANALISIS KOMPARATIF ARSITEKTUR FUSI STEREO UNTUK ESTIMASI POSE 6D DARI
VIDEO STEREOSKOPIK**

Disusun oleh:

MORENO KRISTOVAN (32210013)

Telah disetujui dan diterima sebagai salah satu karya ilmiah mahasiswa yang bersangkutan pada
FAKULTAS TEKNOLOGI DAN DESAIN - Program Studi INFORMATIKA Universitas Bunda Mulia

Tangerang, 04 September 2025

Mengetahui
Ketua Program Studi



Agustinus Fritz Wijaya, S.Kom., M.Cs.

Dosen Pembimbing



Dionisia Bhisetya Rarasati, S.Kom., M.T.I.

PERNYATAAN

Saya menyatakan bahwa Skripsi yang berjudul **ANALISIS KOMPARATIF ARSITEKTUR FUSI STEREO UNTUK ESTIMASI POSE 6D DARI VIDEO STEREOSKOPIK**, sepenuhnya karya saya sendiri. Tidak ada bagian di dalamnya yang merupakan plagiat dari karya orang lain dan saya tidak melakukan penjiplakan atau pengutipan dengan cara-cara yang tidak sesuai dengan etika keilmuan yang berlaku dalam masyarakat keilmuan. Atas pernyataan ini, saya siap menanggung resiko / sanksi yang dijatuhkan kepada saya, atau ada klaim dari pihak lain terhadap keaslian karya saya ini.

Tangerang, 13 Agustus 2025

Yang membuat pernyataan,



Moreno Kristovan

Pernyataan Hasil Sidang Tugas Akhir/Skripsi*
Tanggal Ujian : 13-08-2025

Kami yang bertanda tangan di bawah ini, menyatakan bahwa mahasiswa:

Nama : MORENO KRISTOVAN

Nim : 32210013

Program Studi : INFORMATIKA

Judul Skripsi : ANALISIS KOMPARATIF ARSITEKTUR FUSI STEREO UNTUK ESTIMASI
POSE 6D DARI VIDEO STEREOSKOPIK

Telah dinyatakan **L U L U S** Sidang Pendadaran

Nama

Ketua Penguji : Evasaria Magdalena Sipayung, S.T., M.T.

Tanda Tangan



Penguji I : Fergie Joanda Kaunang, S.Kom., M.S



Penguji II : Bhustomy Hakim, S.S.I., M.Eng.



ABSTRAK

Estimasi pose 6D menggunakan kamera stereo menghadapi tantangan dalam pemilihan arsitektur fusi yang optimal. Penelitian ini melakukan analisis komparatif terhadap tiga arsitektur fusi stereo untuk menentukan pendekatan paling efektif dalam estimasi pose 6D objek tunggal, dengan fokus pada keseimbangan akurasi dan efisiensi. Metodologi ini membandingkan tiga arsitektur berbasis backbone ConvNeXt V2 NANO: model baseline (tanpa fusi), jaringan Siamese (shared-weight), dan fusi dini (early fusion) dengan input 6-channel. Untuk stabilitas, digunakan representasi rotasi kontinu 6D, dan dataset dilabeli menggunakan marker ArUco sebagai ground truth. Hasil evaluasi menunjukkan arsitektur fusi dini secara konsisten mencapai akurasi terbaik dengan nilai Root Mean Square Error (RMSE) translasi dan rotasi terendah. Model ini efektif memanfaatkan disparitas stereo sejak awal pemrosesan, sementara model baseline menunjukkan efisiensi inferensi tertinggi. Disimpulkan bahwa strategi fusi dini menawarkan keseimbangan terbaik antara akurasi dan efisiensi, menjadikannya prospektif untuk aplikasi real-time pada perangkat terbatas. Penelitian ini berkontribusi pada pengembangan solusi estimasi pose 6D berbasis kamera stereo yang efisien tanpa bergantung pada sensor mahal seperti LiDAR. Sebuah antarmuka Streamlit juga dikembangkan untuk visualisasi interaktif.

Kata Kunci: ConvNeXt, Estimasi Pose 6D, Fusi Stereo, Visi Komputer

ABSTRACT

6D pose estimation using stereo cameras faces the challenge of selecting an optimal fusion architecture. This research conducts a comparative analysis of three stereo fusion architectures to determine the most effective approach for single-object 6D pose estimation, focusing on the balance between accuracy and efficiency. The methodology compares three architectures based on the ConvNeXt V2 NANO backbone: a baseline model (no fusion), a Siamese network, and early fusion with a 6-channel input. For stability, a continuous 6D rotation representation is used, and the dataset is labeled using ArUco markers as ground truth. Evaluation results show that the early fusion architecture consistently achieves the best accuracy, with the lowest translation and rotation Root Mean Square Error (RMSE) values. This model effectively utilizes stereo disparity from the initial processing stages, while the baseline model demonstrates the fastest inference. It is concluded that the early fusion strategy offers the best balance between accuracy and efficiency, making it a prospective solution for real-time applications on resource-constrained devices. This research contributes to the development of efficient stereo camera-based 6D pose estimation solutions without relying on expensive sensors like LiDAR. A Streamlit interface was also developed for interactive visualization.

Keywords: *ConvNeXt, 6D Pose Estimation, Stereo Fusion, Computer Vision*

PRAKATA

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan penyusunan skripsi yang berjudul **“ANALISIS KOMPARATIF ARSITEKTUR FUSI STEREO UNTUK ESTIMASI POSE 6D DARI VIDEO STEREOSKOPIK”**. Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana pada Program Studi Informatika, Fakultas Teknologi dan Desain, Universitas Bunda Mulia.

Dalam proses penyusunan skripsi ini, penulis telah menerima banyak bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih yang sebesar-besarnya kepada:

1. **Bapak Doddy Surja Bajuadji, S.E., M.M., Ph.D.**, selaku Rektor Universitas Bunda Mulia.
2. **Bapak Howard S. Giam, S.E., Ak., M.B.A.**, selaku Pelaksana Harian Rektor (PHR) Universitas Bunda Mulia.
3. **Ibu Kandi Sofia Senastri Dahlan, M.B.A., Ph.D.**, selaku Wakil Rektor Bidang Akademik Universitas Bunda Mulia.
4. **Ibu Dr. Shanty Sudarji, S.Psi., M.Psi.**, sebagai Director dari Universitas Bunda Mulia Kampus Serpong.

5. **Bapak Dr. Fransiskus Adikara, S.Kom., M.MSi.**, selaku Dekan Fakultas Teknologi dan Desain Universitas Bunda Mulia.
6. **Bapak Agustinus Fritz Wijaya, S.Kom, M.Cs.**, selaku Ketua Program Studi Informatika Universitas Bunda Mulia.
7. **Ibu Dionisia Bhisetya Rarasati, S.Kom., M.T.I.**, selaku dosen pembimbing, atas bimbingan, arahan, dan masukan yang sangat berarti selama proses penelitian dan penulisan skripsi ini.
8. Seluruh dosen dan staf pengajar di Fakultas Teknologi dan Desain, atas ilmu dan pengetahuan yang telah diberikan selama masa perkuliahannya.
9. Orang tua dan keluarga, atas doa, semangat, dan dukungan moral maupun materiil yang tidak pernah berhenti mengalir.
10. Teman-teman seperjuangan dan semua pihak yang tidak dapat disebutkan satu per satu, yang telah memberikan bantuan dan semangat selama proses penyusunan skripsi ini.

Penulis menyadari bahwa skripsi ini masih jauh dari sempurna. Oleh karena itu, segala bentuk kritik dan saran yang membangun sangat penulis harapkan untuk perbaikan di masa mendatang. Akhir kata, penulis berharap skripsi ini dapat bermanfaat bagi pembaca dan pihak-pihak yang berkepentingan.

Tangerang , 13 Agustus 2025

Moreno Kristovan

DAFTAR ISI

ABSTRAK	v
ABSTRACT	vi
PRAKATA	vii
DAFTAR ISI	ix
DAFTAR GAMBAR	xii
DAFTAR TABEL	xiii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Ruang Lingkup	4
1.4 Tujuan Penelitian	5
1.5 Manfaat Penelitian	6
1.6 Metodologi Penelitian	7
1.7 Sistematika Penulisan	8
BAB 2 LANDASAN TEORI	9
2.1 Digital Camera	9
2.2 Stereoscopic Vision	10
2.2.1 Video Stereoskopik (<i>Stereo Video</i>)	12
2.3 6DoF	13
2.3.1 6D rotation representation for continuous tracking	15
2.4 ArUco Markers	16
2.4.1 Translation <i>Vector</i> dan <i>rotation vector</i>	18
2.5 Deep Learning	19
2.6 Arsitektur Model Fusi stereo	20
2.6.1 Tanpa Fusi (<i>Baseline</i>)	21
2.6.2 Jaringan Saraf Siamese	21
2.6.3 Fusi Dini (Early Fusion)	22
2.7 ConvNext	23

2.7.1 ConvNext V2 Nano	24
2.8 Hyperparameter	24
2.9 Evaluasi arsitektur	27
2.10 Penelitian Terdahulu	30
BAB 3 Analisis dan perancangan	40
3.1 Rekayasa Perencanaan	40
3.1.1 Kebutuhan Fungsional	40
3.1.2 Kebutuhan Non-Fungsional	41
3.2 Pemilihan model dan tahapan pemecah masalah	41
3.2.1 Koreksi Distorsi Lensa dan Kalibrasi Kamera	42
3.2.2 Pemilihan versi Model ConvNeXt	42
3.3 Perancangan Proses	44
3.3.1 Kalibrasi Kamera	46
3.3.2 Akuisisi Data dan Labeling	48
3.3.3 Pembangunan Model	50
3.3.3.1 Tanpa Fusi (<i>Baseline</i>)	52
3.3.3.2 Jaringan Saraf Siamese (<i>Siamese Neural Network</i>)	52
3.3.3.3 Fusi Dini (<i>Early Fusion</i>)	53
3.3.4 Model Evaluation	54
3.4 Perancangan Tampilan Aplikasi	56
3.4.1 Home Page	56
3.4.2 Documentation	58
3.5 Perancangan Tracking Object	59
3.6 Jadwal Pengerjaan	62
BAB 4 Implementasi	63
4.1 Implementasi Penelitian	63
4.2 Implementasi Metode	69
4.2.1 Pemrosesan Dataset dan Arsitektur Folder	70
4.2.1.1 Properti Dataset	74
4.2.2 Hyperparameter Tuning	75
4.2.3 Arsitektur Model	77

4.2.4 Konversi ke TorchScript	81
4.2.5 Sistem Evaluasi dan Metrik Evaluasi	82
4.2.6 Hyperparameter tuning	85
4.2.7 Implementasi Streamlit	87
4.3 Hasil Akhir dan Analisis Komparatif	92
4.3.1 Analisis Performa Agregat	93
4.3.2 Analisis Kualitatif pada Sampel Prediksi	94
4.3.3 Ringkasan Analisis	96
BAB 5 KESIMPULAN DAN SARAN	97
5.1 Kesimpulan	97
5.2 Saran	99
DAFTAR PUSTAKA	103

DAFTAR GAMBAR

Gambar 2.1: Diagram Digital Kamera	9
Gambar 2.2: Diagram Kamera Stereoskopik	12
Gambar 2.3: Contoh Citra Stereoskopik	13
Gambar 2.4: 6 Degrees of Freedom (6DoF)	15
Gambar 2.5: ArUco	17
Gambar 2.6: Nvidia-SMI	29
Gambar 3.1: Metode Riset	45
Gambar 3.2: Kalibrasi Kamera	46
Gambar 3.3: Rektifikasi Stereo	47
Gambar 3.4: Akuisisi data and Labeling	48
Gambar 3.5: Citra Pada Dataset	50
Gambar 3.6: Arsitektur Tanpa Fusi (Baseline)	52
Gambar 3.7: Arsitektur Jaringan Saraf Siamese (<i>Siamese Neural Network</i>) ...	53
Gambar 3.8: Arsitektur Fusi Dini(Early Fusion)	54
Gambar 3.9: Metode Evaluasi Model	55
Gambar 3.10: Streamlit Home Page GUI	57
Gambar 3.11: Streamlit Home Page GUI	58
Gambar 4.1: Implementasi Streamlit	63
Gambar 4.2: Implementasi HomePage	64
Gambar 4.3: Implementasi Dashboard	65
Gambar 4.4: Implementasi Dashboard Dataset Info	66
Gambar 4.5: Implementasi Dashboard, Dataset MD	67
Gambar 4.6: Implementasi Dashboard, Model Evaluation	68
Gambar 4.7: Implementasi Documentation	69

DAFTAR TABEL

Tabel 3.1: ArUco Marker Kubus	60
Tabel 3.2: Jadwal Pengerjaan	62
Tabel 4.1: Properti Dataset	75
Tabel 4.2: Arsitektur Regression Head	80
Tabel 4.3: Data Mentah Hasil Validasi Komparatif	93
Tabel 4.4: Analisis Persentase Peningkatan Performa	94

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Estimasi pose 6D, yang bertugas untuk menentukan posisi tiga dimensi (x, y, z) dan orientasi (yaw, pitch, roll) sebuah objek, telah menjadi teknologi pilar yang fundamental dalam kemajuan berbagai bidang modern. Mulai dari lengan robot yang membutuhkan presisi tinggi di lingkungan industri, aplikasi *augmented reality* (AR) yang memadukan dunia digital dan fisik secara mulus, hingga sistem navigasi pada kendaraan otonom, semua sangat bergantung pada kemampuan sistem untuk memahami pose objek secara akurat dan dalam waktu nyata (*real-time*) [1]. Seiring dengan meningkatnya kompleksitas aplikasi-aplikasi ini, muncul sebuah tantangan krusial yaitu bagaimana mengembangkan solusi estimasi pose yang tidak hanya presisi, tetapi juga efisien secara komputasi dan dapat diimplementasikan menggunakan sensor yang lebih terjangkau, sehingga membuka jalan bagi inovasi yang lebih luas dan dapat diakses. Meskipun metode berbasis visi monokular terus berkembang, visi stereoskopik menawarkan keuntungan fundamental dalam persepsi kedalaman yang langsung, yang sangat krusial untuk aplikasi robotika presisi [2].

Oleh karena itu, visi stereoskopik yang menggunakan dua kamera untuk meniru penglihatan binokular manusia muncul sebagai alternatif yang sangat

menjanjikan dibandingkan sensor yang lebih mahal seperti LiDAR atau ToF [3]. Keunggulan utamanya terletak pada kemampuannya untuk mengekstraksi informasi kedalaman secara langsung dari perbedaan paralaks antara gambar kiri dan kanan. Namun, potensi penuh dari data stereo ini sangat bergantung pada sebuah faktor krusial yang seringkali menjadi tantangan: **bagaimana informasi dari kedua pandangan tersebut digabungkan atau "difusikan"** dalam sebuah arsitektur *deep learning*. Hingga saat ini, belum ada konsensus yang jelas mengenai strategi fusi mana, apakah menggabungkan data mentah di awal (*early fusion*) atau menyatukan fitur-fitur yang telah diekstraksi di akhir (*late fusion*), yang secara definitif paling unggul [4]. Celah dalam pemahaman empiris mengenai untung-rugi dari berbagai arsitektur fusi inilah yang menjadi motivasi utama dan fondasi dari penelitian ini.

Pentingnya penelitian ini terletak pada upaya untuk mengisi celah pengetahuan tersebut secara sistematis. Tanpa analisis perbandingan yang terukur, pemilihan arsitektur fusi menjadi subjektif dan berisiko menghasilkan sistem yang suboptimal, baik dari segi akurasi maupun efisiensi. Memahami arsitektur mana yang menawarkan keseimbangan terbaik akan secara langsung berkontribusi pada pengembangan sistem estimasi pose 6D yang lebih andal, cepat, dan hemat sumber daya. Hal ini sangat vital untuk aplikasi yang sangat sensitif terhadap latensi seperti pelacakan objek pada AR, atau untuk implementasi pada perangkat dengan keterbatasan komputasi (*edge devices*), di mana setiap milidetik dan megabyte memori yang dihemat dapat menentukan keberhasilan implementasi di

dunia nyata. Untuk menjawab tantangan ini, penelitian ini mengusulkan sebuah **analisis komparatif yang terstruktur** terhadap tiga arsitektur fusi stereo yang fundamental: model *baseline* sebagai titik acuan, jaringan Siamese (*Siamese Neural Network*) sebagai representasi fusi akhir, dan model *early fusion 6-channel* sebagai representasi fusi dini. Agar perbandingan antar arsitektur ini dapat berjalan adil dan valid, diperlukan sebuah *backbone* pengekstrak fitur (*feature extractor*) yang konsisten, kuat, dan efisien. Untuk tujuan ini, dipilih **ConvNeXt V2 NANO**, sebuah arsitektur konvolusional modern yang dikenal akan performa dan efisiensinya yang tinggi dalam berbagai tugas visi komputer. Dengan menggunakan *backbone* yang sama untuk semua varian, penelitian ini dapat secara efektif mengisolasi variabel dan mengukur dampak murni dari setiap strategi fusi.

Melalui pendekatan komparatif ini, penelitian ini diharapkan dapat memberikan kontribusi signifikan berupa bukti empiris yang jelas mengenai arsitektur fusi stereo mana yang paling optimal untuk tugas estimasi pose 6D. Hasilnya tidak hanya akan menjadi panduan praktis bagi para pengembang dan peneliti dalam merancang sistem visi yang lebih baik, tetapi juga memperkaya khazanah keilmuan di bidang visi komputer. Pada akhirnya, penelitian ini bertujuan untuk mendorong terciptanya solusi estimasi pose 6D berbasis kamera stereo yang lebih akurat, efisien, dan dapat diakses, tanpa harus bergantung pada sensor yang mahal dan kompleks.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, penelitian ini berusaha menjawab pertanyaan-pertanyaan berikut:

- A. Bagaimana perbandingan performa antara tiga arsitektur fusi stereo yang diuji, yaitu model *baseline* (tanpa fusi), jaringan Siamese (*Siamese Neural Network*), dan fusi dini (*early fusion*), dalam tugas estimasi pose 6D?
- B. Dari ketiga arsitektur tersebut, manakah yang mampu menghasilkan tingkat akurasi tertinggi dalam estimasi translasi dan rotasi, yang diukur menggunakan metrik *Root Mean Square Error* (RMSE) ?
- C. Bagaimana *trade-off* yang ditawarkan oleh setiap arsitektur antara akurasi prediksi dengan efisiensi sumber daya komputasi, khususnya terkait kecepatan inferensi dan penggunaan memori (VRAM)?
- D. Apa saja keterbatasan teknis dan karakteristik unik dari masing-masing pendekatan fusi yang teridentifikasi selama proses evaluasi?

1.3 Ruang Lingkup

Ruang lingkup penelitian ini dibatasi pada:

- A. **Objek singular:** Fokus penelitian adalah pada satu objek dalam satu frame, tanpa mempertimbangkan deteksi multi-objek.

- B. **Data input:** Menggunakan video stereoskopik yang diambil dari kamera stereoskopik standar.
- C. **Arsitektur:** Menggunakan tiga arsitektur, *Baseline* (tanpa fusi), Jaringan Siamese (*Siamese Neural Network*), dan Fusi Dini(*Early Fusion*).
- D. **Evaluasi:** Melibatkan pengukuran akurasi, kecepatan proses, dan efisiensi komputasi.
- E. **Aplikasi:** Studi kasus simulasi dilakukan dalam lingkungan terkontrol.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

- 1) Melakukan analisis perbandingan performa secara kuantitatif terhadap tiga arsitektur fusi stereo yang berbeda: tanpa fusi(*baseline*), jaringan Siamese (*Siamese Neural Network*), dan fusi dini (*early fusion*).
- 2) Mengidentifikasi arsitektur fusi stereo yang mampu menghasilkan tingkat akurasi estimasi pose 6D tertinggi, berdasarkan pengukuran metrik *Root Mean Square Error* (RMSE) untuk translasi dan rotasi.
- 3) Menganalisis keseimbangan (*trade-off*) antara akurasi dan efisiensi komputasi (kecepatan inferensi dan penggunaan VRAM) dari setiap arsitektur untuk memberikan panduan dalam pemilihan model.
- 4) Menghasilkan bukti empiris yang dapat menjadi dasar rekomendasi untuk implementasi arsitektur fusi stereo pada aplikasi visi komputer yang membutuhkan performa *real-time*.

1.5 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah:

- A. Bagi Penulis, Penelitian ini memberikan pengalaman dan pemahaman mendalam mengenai implementasi dan evaluasi arsitektur deep learning untuk visi komputer. Secara praktis, penulis memperoleh kompetensi teknis dalam merancang, melatih, dan menganalisis model estimasi pose 6D, mulai dari tahap kalibrasi kamera, pembuatan dataset, hingga pengembangan aplikasi interaktif menggunakan Streamlit.
- B. Bagi Industri dan Masyarakat, Hasil penelitian ini menawarkan panduan berbasis bukti yang dapat langsung diaplikasikan oleh industri untuk mengembangkan produk teknologi yang lebih efisien dan terjangkau. Bagi perusahaan yang bergerak di bidang robotika, augmented reality, atau otomasi, temuan ini membantu mempercepat pengambilan keputusan teknis dalam memilih arsitektur yang paling sesuai dengan kebutuhan produk, baik yang mengutamakan akurasi presisi maupun kecepatan real-time pada perangkat dengan sumber daya terbatas.

C. Bagi Peneliti Selanjutnya, Penelitian ini menyediakan data perbandingan (benchmark) yang transparan dan terdokumentasi dengan baik mengenai performa tiga arsitektur fusi stereo fundamental (baseline, early fusion, dan late fusion). Hasil ini dapat menjadi referensi dasar dan titik awal yang kuat bagi para peneliti lain yang ingin mengeksplorasi lebih lanjut tentang optimasi arsitektur, penggunaan dataset yang lebih kompleks, atau penerapan pada skenario deteksi multi-objek.

1.6 Metodologi Penelitian

Penelitian ini menerapkan metodologi eksperimental kuantitatif untuk mengevaluasi dan membandingkan secara sistematis tiga arsitektur fusi stereo untuk estimasi pose 6D. Proses penelitian dimulai dari tahap kalibrasi kamera stereo untuk mengoreksi distorsi, dilanjutkan dengan akuisisi data video dan pembuatan dataset berannotasi ground truth menggunakan marker ArUco. Inti dari penelitian ini adalah pelatihan dan evaluasi tiga pendekatan arsitektur yang berbeda yaitu baseline (tanpa fusi), Jaringan Siamese (*Siamese Neural Network*), dan fusi dini 6-channel (early fusion) di atas backbone ConvNeXt V2 NANO yang konsisten untuk memastikan perbandingan yang adil. Kinerja setiap arsitektur diukur berdasarkan metrik akurasi, seperti Root Mean Square Error (RMSE), serta efisiensi komputasi untuk menentukan arsitektur yang paling optimal.

1.7 Sistematika Penulisan

Berikut berisi penjelasan isi dari bab-bab yang terdapat dalam penelitian:

BAB 1 PENDAHULUAN

Bab ini mencakup latar belakang, rumusan masalah, tujuan, manfaat, ruang lingkup, metodologi penelitian, dan sistematika penulisan.

BAB 2 LANDASAN TEORI

Bab ini menjelaskan teori terkait, termasuk citra digital, kamera stereo, kalibrasi, perhitungan disparitas, serta metode deep learning seperti ConvNeXt untuk estimasi 6D pose.

BAB 3 METODOLOGI PENELITIAN

Bab ini menguraikan proses penelitian, mulai dari pengumpulan data, preprocessing, pelatihan model, hingga teknik evaluasi performa estimasi 6D pose.

BAB 4 IMPLEMENTASI DAN EVALUASI

Bab ini membahas penerapan model serta pengujian akurasi estimasi 6D pose menggunakan matrik evaluasi yang relevan.

BAB 5 KESIMPULAN DAN SARAN

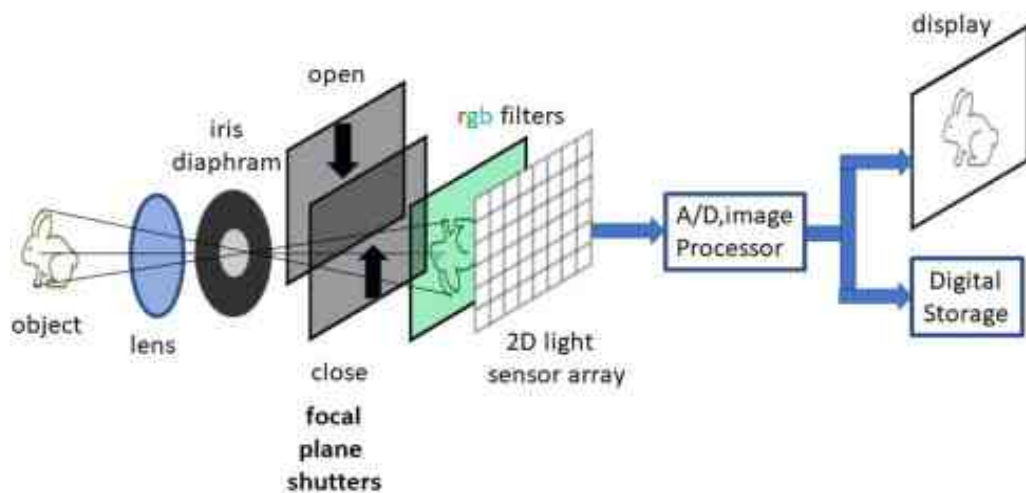
Bab ini berisi kesimpulan penelitian dan saran untuk pengembangan lebih lanjut.

BAB 2

LANDASAN TEORI

2.1 Digital Camera

Digital Kamera adalah perangkat keras elektronik yang bekerja untuk menangkap gelombang elektromagnetik (cahaya) menjadi gelombang listrik digital.



Gambar 2.1: Diagram Digital Kamera

Sebuah kamera digital seperti yang digambarkan pada Gambar 2.1 memiliki komponen utama seperti lensa, aperture (diatur oleh diafragma iris), dan array fotoelektrik digital dengan elemen $m \times n$. Lensa dipilih untuk memfokuskan objek ke bidang gambar, yang terdiri dari filter mosaik warna 2D untuk mengubah elemen individu dalam array agar dapat mendeteksi warna merah, biru, atau hijau. Array ini kemudian mengonversi cahaya yang masuk menjadi sinyal listrik, yang

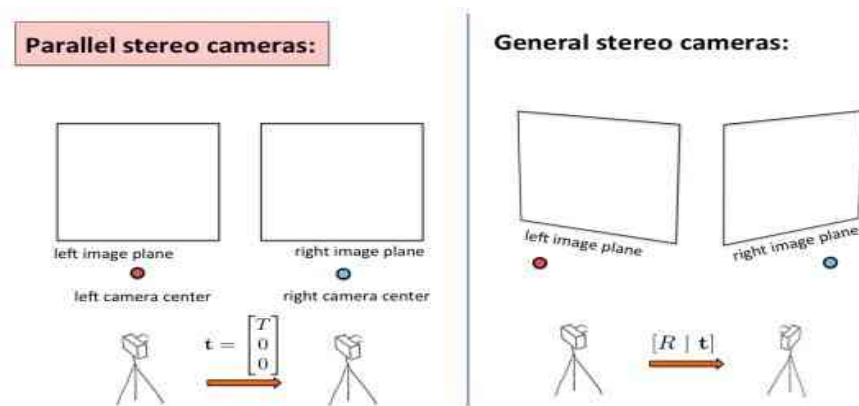
dirakit menjadi matriks $m \times n$ nilai warna interpolasi. Nilai-nilai ini kemudian ditampilkan dan disimpan secara digital dalam format yang dipilih.

Kamera digital kelas atas saat ini memiliki sekitar 50 juta elemen dalam *array*-nya. Tiap kamera digital memiliki sarana kontrol pencahayaan seperti *F-Stop*, ISO, dan *Shutter Speed* serta sarana kontrol pada warna, *Saturation*, *White Balance*(*Color Temperature*)[5]. Karena array sensor pada kamera memiliki rentang dinamis atau tingkat sensitivitas cahaya, jumlah cahaya yang masuk selama pencahayaan idealnya diatur pada tingkat tengah untuk memaksimalkan penggunaan rentang tersebut. Penyesuaian pencahayaan dilakukan dengan mengubah kecepatan rana dan ukuran aperture. Aperture mengatur jumlah cahaya yang melewati diafragma iris atau dikenal dengan istilah "F-stop", sementara durasi pencahayaan dikendalikan oleh dua rana bidang fokus: satu membuka untuk membiarkan cahaya masuk, dan satu lagi menutup untuk menghentikan pencahayaan. Namun, pengaturan ini sangat bergantung pada pemahaman pengguna terhadap fitur kamera. Kurangnya pengetahuan mengenai pengaturan tersebut dapat menyebabkan hasil foto digital tidak sesuai harapan, misalnya eksposur berlebih atau kurang, yang sejalan dengan temuan bahwa kualitas citra digital sangat dipengaruhi oleh spesifikasi kamera dan kemampuan pengguna dalam mengoperasikannya[6].

2.2 Stereoscopic Vision

Kamera stereo adalah jenis kamera yang dirancang untuk meniru penglihatan stereoskopik manusia dengan menggunakan dua atau lebih kamera

untuk menangkap pemandangan yang sama dari sudut pandang yang sedikit berbeda. Biasanya, dua kamera ini disusun dalam konfigurasi binokular untuk memungkinkan estimasi kedalaman melalui analisis paralaks, yaitu perbedaan koordinat proyeksi suatu titik target pada bidang dua kamera. Proses ini melibatkan triangulasi, di mana hubungan geometris antara kamera, jarak antar kamera (baseline), dan koordinat target digunakan untuk menghitung informasi kedalaman. Semakin panjang baseline, semakin jauh jarak yang dapat diukur.



Gambar 2.2: Diagram Kamera Stereoskopik

Kamera stereo memungkinkan estimasi kedalaman untuk membangun gambar 3D, tetapi prosesnya membutuhkan perhitungan intensif untuk menentukan kedalaman setiap piksel secara akurat. Sistem ini mengandalkan disparitas antara dua gambar untuk menghasilkan posisi spasial, menjadikannya sangat berguna dalam aplikasi seperti kendaraan otonom dan robotika. Namun, konfigurasi dan kalibrasi kamera ini cukup rumit, terutama jika menggunakan lebih dari dua kamera. Sistem dengan tiga atau lebih kamera bekerja serupa dengan konfigurasi binokular tetapi meningkatkan kompleksitas dengan tujuan meningkatkan akurasi estimasi.

2.2.1 Video Stereoskopik (*Stereo Video*)

Video stereoskopik secara umum adalah penerapan prinsip visi stereo pada gambar bergerak. Teknik ini menggunakan sistem dua kamera yang merekam sebuah adegan secara bersamaan dari dua sudut pandang yang sedikit berbeda,

menghasilkan dua aliran video yang sinkron: satu untuk "mata" kiri dan satu lagi untuk "mata" kanan.



Gambar 2.3: Contoh Citra Stereoskopik

Ketika kedua aliran video ini ditampilkan, disparitas atau perbedaan halus antara frame kiri dan kanan menciptakan ilusi kedalaman (3D). Dalam konteks penelitian ini, setiap frame dari video stereoskopik menyediakan sepasang gambar yang menjadi input data mentah. Pasangan gambar inilah yang akan diolah oleh berbagai arsitektur fusi stereo untuk mengekstraksi informasi spasial yang diperlukan untuk melakukan estimasi pose 6D.

2.3 6DoF

6DoF (6 Degrees of Freedom) merujuk pada enam jenis gerakan atau pergerakan yang dapat dilakukan oleh objek dalam ruang tiga dimensi, yang telah

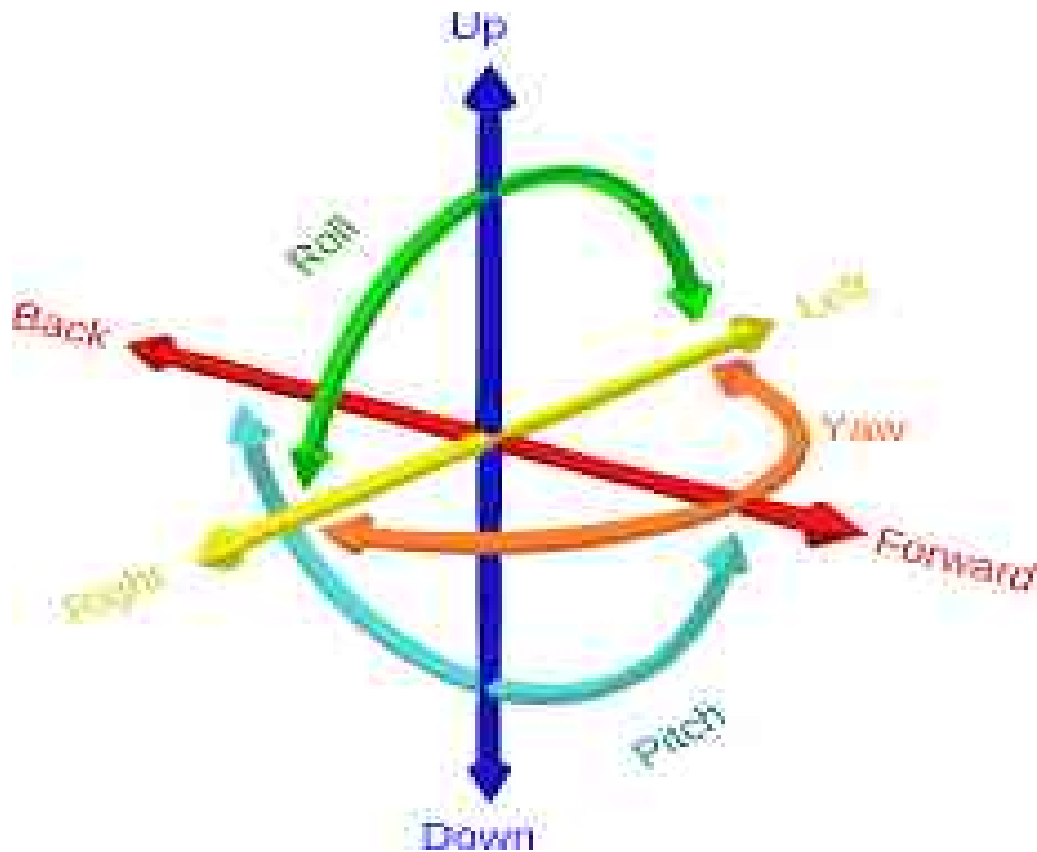
menjadi masalah fundamental dalam berbagai bidang teknologi seperti robotika dan augmented reality [8]. Istilah ini digunakan untuk menggambarkan kemampuan objek atau perangkat untuk bergerak secara bebas di ruang 3D, baik dalam konteks fisik maupun virtual. Enam derajat kebebasan ini terdiri dari tiga gerakan translasi dan tiga gerakan rotasi:

A. Translasi:

- a. **Gerakan maju/mundur** (forward/backward): Gerakan objek di sepanjang sumbu Z (kedalaman).
- b. **Gerakan kiri/kanan** (left/right): Gerakan objek di sepanjang sumbu X (lebar).
- c. **Gerakan atas/bawah** (up/down): Gerakan objek di sepanjang sumbu Y (tinggi).

B. Rotasi(Euler Angles):

- a. **Pitch** (rotasi sekitar sumbu X): Putaran objek ke atas atau ke bawah, seperti kepala yang mengangguk.
- b. **Yaw** (rotasi sekitar sumbu Y): Putaran objek ke kiri atau ke kanan, seperti kepala yang menggeleng.
- c. **Roll** (rotasi sekitar sumbu Z): Putaran objek searah atau berlawanan arah jarum jam, seperti gerakan kepala saat mengangguk ke samping.



Gambar 2.4: 6 Degrees of Freedom (6DoF)

2.3.1 6D Rotation Representation For Continuous Tracking

6D rotation representation for continuous tracking adalah metode untuk merepresentasikan rotasi 3D secara kontinu, stabil, dan bebas dari masalah gimbal lock. Representasi ini menggunakan enam nilai (6 dimensi) yang berasal dari matriks rotasi 3×3 yang valid dan ortogonal. Metode ini dirancang Zhou, et al., khusus untuk digunakan neural network, karena model perlu belajar rotasi secara stabil dan halus selama proses training[9].

Dalam rotasi 3D, kita ingin merepresentasikan elemen dari grup rotasi $SO(3)$, yaitu semua matriks rotasi 3×3 yang ortogonal dan memiliki determinan $+1$. Namun, langsung memprediksi matriks rotasi lengkap (9 nilai) cukup sulit, karena model harus memastikan hasilnya tetap valid secara geometris (ortogonal dan ternormalisasi).

Representasi rotasi 6D menyederhanakan masalah ini dengan hanya menggunakan dua buah vector 3D (total 6 angka), yang akan menjadi dua kolom pertama dari matriks rotasi. Kolom ketiga dapat dihitung menggunakan cross product antara dua kolom pertama. Dengan cara ini, hasil akhirnya tetap berupa matriks rotasi yang valid, tetapi lebih mudah dipelajari oleh neural network karena tidak perlu normalisasi eksplisit.

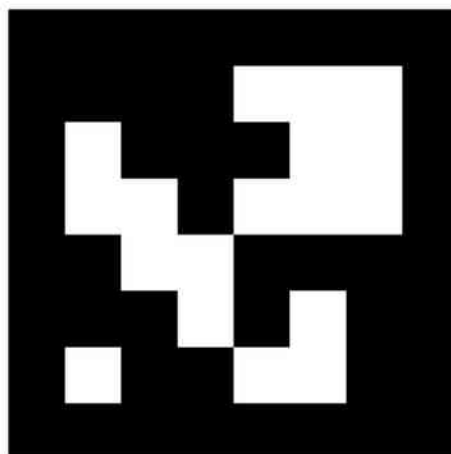
Metode ini terbukti lebih stabil dan akurat dibandingkan representasi rotasi lainnya seperti Euler angles atau quaternion, terutama dalam tugas-tugas seperti pose estimation dan object tracking.

Dibanding Euler angles: Tidak mengalami gimbal lock, tidak diskontinu, dan tidak memerlukan fungsi trigonometri kompleks.

Dibanding quaternion: Lebih mudah untuk dioptimasi karena tidak perlu normalisasi eksplisit (quaternion harus punya norma 1), dan tidak memiliki ambiguitas tanda (quaternion q dan $-q$ merepresentasikan rotasi yang sama).

2.4 ArUco Markers

Menurut Kalaitzakis, et al., ArUco adalah sistem fiducial marker berbasis visual yang digunakan untuk estimasi pose dalam berbagai aplikasi, seperti robotika, augmented reality (AR), dan Computer Vision. Marker ini berbentuk persegi dan berisi pola biner unik yang memungkinkan sistem komputer mengenali dan mendeteksinya secara akurat dari citra atau video[10]. Dibandingkan dengan metode marker lainnya, ArUco menawarkan integrasi yang mudah dengan computer vision library seperti OpenCV, serta performa deteksi yang baik dalam berbagai kondisi pencahayaan dan sudut pandang.



Gambar 2.5: ArUco

Proses deteksi ArUco dimulai dengan pencarian kontur berbentuk persegi dalam gambar untuk mengidentifikasi kandidat marker. Setelah marker terdeteksi, sistem akan mengekstrak ID biner yang dikodekan dalam pola marker tersebut dan memverifikasinya terhadap set kode yang valid. ArUco juga menyediakan mekanisme koreksi kesalahan untuk memastikan akurasi pembacaan walaupun terdapat noise atau distorsi dalam citra. Setelah marker dikenali, algoritma estimasi pose digunakan untuk menghitung posisi dan orientasi marker dalam ruang tiga dimensi dengan memanfaatkan parameter kalibrasi kamera.

Keunggulan utama ArUco terletak pada kemudahan penggunaannya, dukungan melalui OpenCV, serta kemampuan deteksi real-time yang efisien bahkan pada perangkat dengan kemampuan komputasi terbatas. Dibandingkan dengan fiducial marker lain seperti AprilTag, ArUco banyak digunakan karena fleksibilitasnya dalam membuat marker sendiri, kemudahan integrasi, standarisasi, serta dokumentasi yang luas. Oleh karena itu, ArUco menjadi pilihan populer dalam penelitian, pengembangan prototipe, dan aplikasi industri yang membutuhkan sistem deteksi marker yang cepat, akurat, dan stabil.

2.4.1 Translation Vector Dan Rotation Vector

Ketika fungsi estimasi pose dari OpenCV berhasil mendeteksi sebuah marker ArUco dalam sebuah citra dan diberikan parameter kalibrasi kamera, fungsi tersebut akan menghasilkan dua vektor utama yang mendefinisikan pose 6D dari marker tersebut relatif terhadap sistem koordinat kamera: translation vector (*tvec*) dan rotation vector (*rvec*).

Translation Vector (*tvec*) adalah sebuah vektor tiga elemen (x, y, z) yang merepresentasikan posisi translasional dari pusat geometris marker dalam ruang tiga dimensi. Vektor ini mendefinisikan jarak dari titik pusat optik kamera ke pusat marker di sepanjang sumbu X , Y , dan Z dari sistem koordinat kamera. Satuan dari *tvec* ini konsisten dengan satuan yang digunakan untuk mendefinisikan ukuran fisik dari marker saat proses estimasi (misalnya, meter atau sentimeter).

Rotation Vector (rvec), di sisi lain, adalah sebuah vektor tiga elemen yang merepresentasikan orientasi atau rotasi marker. Vektor ini menggunakan representasi sumbu-sudut (*axis-angle*), yang juga dikenal sebagai representasi Rodrigues. Arah dari vektor ini mendefinisikan sumbu rotasi di dalam ruang 3D, sementara magnitudo (panjang) dari vektor ini merepresentasikan besaran sudut rotasi (dalam radian) di sekitar sumbu tersebut. Representasi ini lebih ringkas dibandingkan matriks rotasi dan terhindar dari isu gimbal lock yang sering terjadi pada penggunaan Sudut Euler. Untuk keperluan kalkulasi lebih lanjut, *rvec* dapat dikonversi menjadi matriks rotasi 3x3 menggunakan fungsi `cv2.Rodrigues()`.

2.5 Deep Learning

Deep learning adalah *subsection* dari machine learning yang merupakan bagian dari *Artificial Intelligence* (AI), yang berfokus pada pemodelan data menggunakan arsitektur yang menyerupai cara kerja otak manusia, yaitu jaringan saraf tiruan (artificial neural networks)[11]. Istilah

"deep" merujuk pada banyaknya lapisan (layers) yang digunakan dalam jaringan tersebut, biasanya terdiri dari puluhan hingga ratusan lapisan tersembunyi (hidden layers) yang memungkinkan model untuk mempelajari representasi data yang sangat kompleks dan berlapis-lapis. Setiap lapisan dalam jaringan saraf berfungsi untuk mengekstraksi fitur atau pola dari data, mulai dari fitur sederhana di lapisan awal hingga pola yang lebih abstrak di lapisan yang lebih dalam. G. Verma dan R. Doriya menyatakan bahwa, Deep learning telah merevolusi banyak bidang seperti *Facial Recognition*, *Natural Language Processing* (NLP), *Autonomous Vehicles*, dan *Radiographic Medical Diagnosis*, karena kemampuannya untuk belajar langsung dari data mentah dalam jumlah besar tanpa perlu ekstraksi fitur secara manual[11]. Secara khusus dalam visi stereoskopik, penggunaan Convolutional Neural Networks (CNN) telah dieksplorasi untuk menghasilkan *disparity map* yang lebih akurat dibandingkan dengan metode-metode tradisional, terutama pada area dengan sedikit tekstur [12]. Melalui proses pelatihan menggunakan algoritma backpropagation dan optimisasi berbasis gradient descent, deep learning mampu menyesuaikan bobot-bobot koneksi antar neuron agar dapat menghasilkan prediksi atau keputusan yang akurat.

2.6 Arsitektur Model Fusi Stereo

Dalam estimasi pose 6D berbasis visi stereoskopik, strategi penggabungan atau fusi informasi dari pasangan gambar kiri dan kanan merupakan langkah krusial yang menentukan bagaimana model deep learning memahami kedalaman

dan geometri tiga dimensi. Terdapat berbagai pendekatan untuk melakukan fusi ini, yang secara umum dapat dikategorikan berdasarkan pada tahap mana informasi dari kedua sumber digabungkan. Penelitian ini secara spesifik menganalisis dan membandingkan tiga arsitektur fundamental: sebuah model baseline sebagai titik acuan, Jaringan Saraf Siamese (Siamese Neural Network) yang merepresentasikan fusi akhir (late fusion), dan arsitektur fusi dini (early fusion) yang menggabungkan data pada level input [4].

2.6.1 Tanpa Fusi (*Baseline*)

Arsitektur baseline dirancang sebagai titik kontrol untuk mengukur peningkatan performa yang diberikan oleh arsitektur fusi yang lebih kompleks. Dalam pendekatan ini, tidak ada mekanisme fusi data yang canggih yang diterapkan. Sebaliknya, citra input mentah dari kamera stereo, yang memiliki dimensi asli (misalnya, 1920x480 piksel), diambil dan secara langsung diubah ukurannya menjadi dimensi input yang dibutuhkan oleh model, sesuai dengan hyperparameter IMAGE_SIZE (224x224 atau 384x384 piksel). Proses ini secara efektif "memadatkan" kedua pandangan kiri dan kanan ke dalam satu gambar tunggal. Meskipun inputnya berasal dari sumber stereo, model ini memperlakukannya sebagai citra monokular standar. Pengujian terhadap arsitektur ini sangat penting untuk memvalidasi apakah strategi fusi yang lebih rumit benar-benar memberikan keuntungan yang signifikan dibandingkan dengan pendekatan paling sederhana dalam memproses data stereo.

2.6.2 Jaringan Saraf Siamese

Jaringan Saraf Siamese (Siamese Neural Network) adalah arsitektur yang ideal untuk tugas-tugas yang melibatkan perbandingan antara dua input, seperti pada visi stereoskopik. Dalam konteks ini, arsitektur Siamese menggunakan dua cabang jaringan pengekstrak fitur yang identik dan berbagi bobot (shared-weight) untuk memproses gambar kiri dan kanan secara paralel. Fitur-fitur yang diekstraksi dari kedua cabang tersebut kemudian digabungkan pada tahap akhir, biasanya melalui operasi konkatenasi, sebelum dimasukkan ke regression head untuk estimasi pose. Pendekatan ini dikenal sebagai late fusion karena fusi informasi terjadi setelah fitur-fitur tingkat tinggi telah diekstraksi. Mengikuti penjelasan An dan Yan, penggunaan jaringan Siamese efektif untuk menghitung afinitas atau hubungan antara objek visual dalam input yang berbeda [13]. Keunggulan potensial dari arsitektur ini adalah kemampuannya untuk mempelajari representasi fitur yang kaya dari setiap pandangan secara independen sebelum membandingkannya.

2.6.3 Fusi Dini (Early Fusion)

Berbeda dengan pendekatan fusi akhir, arsitektur fusi dini (early fusion) menggabungkan informasi dari pasangan gambar stereo pada level input mentah, sebelum diproses oleh jaringan. Dalam penelitian ini, fusi dini diimplementasikan dengan menumpuk channel warna dari kedua gambar, menciptakan sebuah tensor input tunggal dengan 6 channel (3 channel RGB dari gambar kiri dan 3 channel RGB dari gambar kanan). Input multi-channel ini kemudian diproses melalui satu

alur jaringan. Menurut Alaba et al., strategi early fusion memungkinkan jaringan untuk mengeksplorasi korelasi spasial tingkat rendah antar input sensor sejak lapisan-lapisan awal [4]. Potensi keuntungan dari metode ini adalah efisiensi komputasi karena hanya memerlukan satu backbone, dan kemampuannya untuk mempelajari fitur-fitur gabungan yang mungkin tidak dapat ditangkap oleh arsitektur yang memproses input secara terpisah.

2.7 ConvNext

ConvNeXt adalah model yang di buat oleh tim Facebook AI research di UC Berkeley, Liu, et al., deep learning berbasis Convolutional Neural Network (CNN) yang dirancang untuk menghadirkan kekuatan CNN dalam era dominasi arsitektur Vision Transformer (ViT)[14]. Dikembangkan oleh tim Meta AI, ConvNeXt merupakan hasil modernisasi dari arsitektur ResNet, dengan mengadopsi berbagai teknik dan prinsip desain arsitektur yang biasa digunakan pada Transformer, namun tetap mempertahankan efisiensi khas CNN. Tujuan utama dari ConvNeXt adalah untuk menyaingi performa Transformer modern tanpa mengorbankan kecepatan dan efisiensi komputasi.

Secara umum, ConvNeXt memproses gambar input melalui serangkaian blok konvolusional yang telah diperbarui dengan struktur yang lebih modern, seperti penggunaan depthwise separable convolution, inverted bottleneck, aktivasi GELU, dan layer normalization. Seluruh desainnya disusun untuk mendukung pelatihan dalam skala besar dan memungkinkan model mencapai akurasi tinggi di berbagai tugas visi komputer, mulai dari klasifikasi citra hingga deteksi objek.

Setelah ekstraksi fitur dilakukan di seluruh blok, fitur tersebut dirangkum melalui global average pooling sebelum akhirnya diproses oleh lapisan output yang menghasilkan prediksi.

Liu menyatakan bahwa keunggulan utama ConvNeXt terletak pada kemampuannya menggabungkan performa tinggi dan efisiensi secara seimbang. Model ini menawarkan fleksibilitas dalam hal ukuran dan kompleksitas, dengan varian seperti ConvNeXt-Tiny, Small, Base, hingga Large, yang memungkinkan pengguna menyesuaikan model dengan kebutuhan dan keterbatasan perangkat keras mereka. Karena alasan ini, ConvNeXt menjadi pilihan menarik untuk berbagai aplikasi, termasuk sistem real-time, dan tugas-tugas yang memerlukan latensi rendah namun tetap akurat.

2.7.1 ConvNext V2 Nano

ConvNeXtV2 NANO adalah salah satu varian terbaru dan paling ringan dari keluarga ConvNeXtV2; Dibuat oleh Woo, et al., Dibandingkan dengan ConvNeXt versi pertama, ConvNeXtV2 menghadirkan sejumlah peningkatan, seperti teknik augmentasi yang lebih baik, normalisasi yang lebih stabil, dan kemampuan generalisasi yang ditingkatkan melalui pelatihan pada resolusi tinggi[15].

ConvNeXtV2 NANO dirancang untuk efisiensi maksimum tanpa mengorbankan performa. Meskipun ukurannya kecil, model ini tetap mampu memberikan representasi fitur yang kaya dan akurat, sehingga ideal untuk aplikasi real-time, embedded systems.

2.8 Hyperparameter

Proses pelatihan model deep learning sangat dipengaruhi oleh pemilihan hyperparameter, yaitu parameter konfigurasi yang nilainya ditetapkan sebelum proses pembelajaran dimulai. Pemilihan hyperparameter yang tepat dapat secara signifikan memengaruhi kecepatan konvergensi, stabilitas pelatihan, dan performa akhir dari sebuah model. Dalam konteks analisis komparatif ini, beberapa hyperparameter kunci dieksplorasi untuk memahami dampaknya terhadap kinerja masing-masing arsitektur fusi stereo.

- A. Ukuran Citra Input (Image Size), Ukuran citra input menentukan resolusi data yang diproses oleh model. Penggunaan resolusi yang lebih tinggi memungkinkan model untuk mengakses detail visual yang lebih kaya, yang berpotensi meningkatkan akurasi dalam mendeteksi fitur-fitur halus pada objek. Namun, hal ini juga secara langsung meningkatkan beban komputasi dan kebutuhan memori (VRAM) karena jumlah data yang harus diproses pada setiap lapisan konvolusional menjadi lebih besar. Sebaliknya, resolusi yang lebih rendah mempercepat proses pelatihan dan inferensi, tetapi berisiko kehilangan informasi penting yang dapat menurunkan akurasi.
- B. Ukuran Batch (Batch Size), Ukuran batch adalah jumlah sampel data yang diproses oleh model sebelum bobotnya diperbarui. Ukuran batch yang lebih besar dapat menghasilkan gradien yang lebih stabil dan representatif dari keseluruhan dataset, yang seringkali mempercepat konvergensi. Namun, ini membutuhkan VRAM yang jauh lebih besar. Di

sisi lain, ukuran batch yang lebih kecil membutuhkan lebih sedikit memori tetapi dapat menghasilkan estimasi gradien yang lebih berisik (noisy), yang meskipun dapat membantu model keluar dari minimum lokal, terkadang juga dapat memperlambat atau mengganggu proses konvergensi.

- C. Laju Pembelajaran (Learning Rate), Laju pembelajaran adalah salah satu hyperparameter yang paling krusial karena mengontrol seberapa besar penyesuaian yang dilakukan terhadap bobot model pada setiap iterasi. Menurut Verma dan Doriya, proses pelatihan pada deep learning sangat bergantung pada algoritma optimisasi seperti gradient descent, di mana laju pembelajaran menentukan ukuran langkah pada setiap pembaruan [11]. Laju pembelajaran yang terlalu tinggi dapat menyebabkan model "melompati" titik optimal dan gagal konvergen. Sebaliknya, laju pembelajaran yang terlalu rendah dapat membuat proses pelatihan menjadi sangat lambat dan berisiko terjebak dalam minimum lokal yang suboptimal.
- D. Epoch, epoch merepresentasikan satu siklus lengkap di mana model deep learning telah melihat dan memproses seluruh dataset pelatihan. Jika sebuah dataset terdiri dari 1.000 gambar dan ukuran batch adalah 100, maka dibutuhkan 10 iterasi ($1.000 / 100$) untuk menyelesaikan satu epoch. Jumlah epoch yang dijalankan selama pelatihan menentukan seberapa sering model "belajar" dari keseluruhan data. Jumlah epoch yang terlalu sedikit dapat menyebabkan underfitting, di mana model belum cukup

belajar untuk mengenali pola. Sebaliknya, jumlah epoch yang terlalu banyak dapat menyebabkan overfitting, di mana model mulai menghafal data.

- E. Patience, Patience adalah hyperparameter yang digunakan dalam mekanisme early stopping, sebuah teknik regularisasi untuk mencegah overfitting. Parameter ini menentukan jumlah epoch yang akan ditunggu oleh model tanpa adanya peningkatan performa pada metrik validasi (misalnya, validation loss) sebelum proses pelatihan dihentikan. Jika performa model tidak membaik setelah jumlah epoch yang ditentukan oleh patience, pelatihan akan berhenti secara otomatis, dan bobot model terbaik yang tersimpan akan digunakan sebagai hasil akhir. Ini memastikan bahwa pelatihan tidak berlanjut sia-sia dan mencegah model menjadi terlalu terspesialisasi pada data pelatihan.
- F. Arsitektur Head, Arsitektur head merujuk pada beberapa lapisan terakhir dari jaringan saraf yang bertanggung jawab untuk mengubah fitur-fitur yang diekstraksi oleh backbone menjadi output akhir, dalam kasus ini adalah nilai translasi dan rotasi. Kompleksitas head, seperti jumlah lapisan dan jumlah neuron di setiap lapisan, sangat mempengaruhi kapasitas model untuk mempelajari hubungan antara fitur dan pose. Head yang terlalu sederhana mungkin tidak mampu menangkap hubungan yang kompleks (underfitting), sementara head yang terlalu kompleks berisiko menghafal data pelatihan (overfitting) dan memiliki performa yang buruk pada data yang belum pernah dilihat sebelumnya.

2.9 Evaluasi Arsitektur

Evaluasi kinerja arsitektur dilakukan menggunakan berbagai metrik untuk menilai seberapa baik model melakukan estimasi pose 6D pada data yang tidak terlihat sebelumnya. Tiga metrik utama yang digunakan dalam penelitian ini adalah Root Mean Square Error (RMSE), Inference Time (ms), serta pemakaian VRAM.

- A. RMSE digunakan untuk mengukur seberapa besar rata-rata kesalahan antara prediksi model dan nilai ground truth. Semakin rendah nilai RMSE, semakin akurat prediksi model. Dalam aplikasi nyata, akurasi ini berdampak langsung pada fungsionalitas sistem. Sebagai contoh, dalam penggunaan sebagai alat HID(Human Interface Device), RMSE translasi yang rendah (dalam satuan centimeter) memastikan objek yang dikontrol pada scene 3D dapat direpresentasikan secara akurat.
- B. Inference time (ms) mengukur seberapa cepat model dapat menghasilkan prediksi untuk satu input, biasanya dinyatakan dalam satuan milidetik (ms). Metrik ini sangat penting dalam konteks aplikasi real-time, seperti robotika, augmented reality, atau sistem navigasi otonom, di mana kecepatan respons sistem menjadi faktor krusial. Model dengan akurasi tinggi tetapi waktu inferensi yang lambat mungkin tidak dapat digunakan secara efektif dalam aplikasi langsung. Oleh karena itu, metrik ini dianalisis untuk memastikan model tidak hanya akurat, tetapi juga cukup cepat untuk skenario operasional yang sebenarnya.

C. VRAM mengukur jejak memori (memory footprint) yang dibutuhkan oleh sebuah model pada Graphics Processing Unit (GPU) selama proses pelatihan dan inferensi. Metrik ini sangat penting untuk menilai efisiensi sumber daya dari sebuah arsitektur. Arsitektur dengan penggunaan VRAM yang tinggi mungkin tidak dapat dilatih atau diimplementasikan pada perangkat keras dengan kapasitas memori yang terbatas. Dalam penelitian ini, penggunaan VRAM diukur dengan menggunakan utilitas NVIDIA System Management Interface (nvidia-smi) pada command line. Proses pengukuran dilakukan dengan memantau proses Python yang sedang berjalan dan mencatat nilai pada kolom "GPU Memory Usage" untuk mendapatkan konsumsi VRAM maksimum selama sesi pelatihan.

Selain ketiga metrik tersebut, metrik lain seperti Waktu Pelatihan (Training Time) juga dicatat sebagai pertimbangan untuk mengevaluasi total biaya komputasi yang dibutuhkan untuk melatih setiap model arsitektur.

```

Sun Aug 17 02:18:37 2025
+-----+
| NVIDIA-SMI 575.57.08                  Driver Version: 575.57.08      CUDA Version: 12.9     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M   Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp            Perf             Pwr:Usage/Cap     Memory-Usage | GPU-Util  Compute M.  |
|=====+=====+=====+=====+=====+=====+
| 0    NVIDIA GeForce RTX 3070        On          00000000:01:00:00    On          N/A           |
| 0%    56C            P0              59W / 220W      824MiB / 8192MiB | 0%         Default      |
|                                           MIG M.         N/A           |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
| ID    ID    ID             |          |                     Usage      |
+-----+-----+-----+-----+-----+-----+
| 0     N/A   N/A         1217    G     /usr/lib/xorg/Xorg             311MiB |
| 0     N/A   N/A         2079    G     cinnamon                      86MiB |
| 0     N/A   N/A         2832    G     ...144 --variations-seed-version 131MiB |
| 0     N/A   N/A         47529   G     ...144 --variations-seed-version 46MiB |
| 0     N/A   N/A         48093   G     ...144 --variations-seed-version 85MiB |
| 0     N/A   N/A         64887   G     /usr/lib/zotero/zotero-bin      20MiB |
| 0     N/A   N/A         75618   G     ...144 --variations-seed-version 35MiB |
| 0     N/A   N/A         88926   G     missioncenter                   15MiB |
+-----+

```

Gambar 2.6: Nvidia-SMI

2.10 Penelitian Terdahulu

Penelitian Terdahulu 1	
Judul	Extending 6D Object Pose Estimators for Stereo Vision
Tahun Terbit	2024

Nama Penulis	Thomas Pöllabauer, Jan Emrich, Volker Knauthe, Arjan Kuijper
Jurnal	arXiv
Kesimpulan	Dalam makalah ini, disajikan solusi untuk estimasi jarak berdasarkan stereoskopik dan algoritma YOLO. Algoritma ini menggunakan dua gambar atau dua aliran video yang diperoleh dari dua kamera yang dipisahkan secara horizontal (berbasis stereoskopik). Pengujian secara ekstensif mengonfirmasi bahwa algoritma saya dapat berguna untuk estimasi jarak dalam jarak hingga 20 meter. Algoritma ini dapat digunakan dalam banyak aplikasi Advanced Driver Assistance Systems (ADAS), seperti aplikasi untuk kontrol jelajah adaptif, parkir otomatis, dan lainnya. Diperlihatkan bahwa kamera, sebagai sensor utama, dapat menggantikan LIDAR dalam beberapa situasi. Terkait dengan skenario pengujian ketika objek berada lebih dari 20 meter dari kamera, masalah umumnya terjadi karena kotak pembatas YOLO yang tidak presisi. Setiap piksel kesalahan menyebabkan deviasi yang signifikan dalam hal akurasi pengukuran. Ada beberapa ide dan pendekatan untuk meningkatkan solusi yang disajikan agar menghasilkan estimasi yang lebih akurat. Sebagai contoh, masalah yang disebabkan oleh detektor YOLO dapat diatasi dengan penggunaan detektor yang lebih presisi atau dengan jaringan saraf yang dilatih dengan cara yang berbeda dari yang digunakan dalam penelitian ini.*
Persamaan	Menggunakan kamera stereoskopik untuk estimasi jarak.

Perbedaan	Menggunakan model YOLO sebagai basis deteksi objek. Bounding Box anotasi 2D Tidak mengestimasi Pose 6D

Penelitian Terdahulu 2	
Judul	A Comprehensive Survey of Deep Learning Multisensor Fusion-based 3D Object Detection for Autonomous Driving: Methods, Challenges, Open Issues, and Future Directions
Tahun Terbit	2022
Nama Penulis	Alaba, Simegnew; Gurbuz, Ali; Ball, John
Jurnal	techrxiv
Kesimpulan	This survey is a SOTA DL sensor fusion-based 3D detection methods survey for AD. The commonly used sensors and datasets for AD were reviewed. However, most datasets do not include inclement weather and nighttime driving to represent the real-world driving system. We expect more representative datasets of the real-world driving systems and a balance between classes in the dataset. A more representative dataset that includes different weather and driving scenario is essential to building a robust driving system. A robust simulator that can simulate different weather and driving scenarios may help build a robust model. The DL-based 3D object detection methods are less lightweight to deploy in a real system than 2D equivalents.

	Additionally, there is no agreed way of selecting the best fusion techniques. To drive anywhere, anytime, and in any weather, sensor fusion is the best solution. Generally, 3D object detection methods show significant performance improvement for AD. However, several open issues exist for improving fusion-based models for real-time processing in terms of speed and accuracy. Researchers do not give much attention to sensor failure research and the possible solutions without affecting the overall system if it happens. We also expect 3D object detection research in open-set conditions with domain adaptation or separately.
Persamaan	Melakukan data fusion
Perbedaan	Menggunakan backbone model yang berbeda

Penelitian Terdahulu 3	
Judul	YOLOPose V2: Understanding and Improving Transformer-based 6D Pose Estimation
Tahun Terbit	2023
Nama Penulis	Arul Selvam Periyasamy, Arash Amini, Vladimir Tsaturyan, Sven Behnke
Jurnal	arXiv preprint
Kesimpulan	Kami memperkenalkan YOLOPose, sebuah metode estimasi pose multi-objek satu tahap yang berbasis pada arsitektur Transformer dan menggunakan pendekatan regresi titik kunci (keypoint regression). Model ini secara simultan memprediksi kotak pembatas (bounding boxes), label kelas, vektor translasi, serta koordinat piksel dari titik kunci 3D untuk semua objek yang terdeteksi dalam gambar input. Dengan mengintegrasikan modul RotEst yang dapat dipelajari (learnable), model ini mampu memperkirakan orientasi objek berdasarkan koordinat titik kunci yang diprediksi, menjadikannya sepenuhnya dapat didiferensiasi secara end-to-end. saya mencapai hasil yang sebanding dengan metode state-of-the-art pada dataset YCB-Video yang banyak digunakan, sekaligus menunjukkan

	kemampuan model untuk beroperasi secara real-time.
Persamaan	Mengestimasi orientasi objek 6D
Perbedaan	Memanfaatkan arsitektur Transformer (dengan object queries dan multi-head self-attention) dibandingkan CNN tradisional yang umum digunakan pada pendekatan stereoskopik.

Penelitian Terdahulu 4	
Judul	FAST GDRNPP: Improving the Speed of State-of-the-Art 6D Object Pose Estimation
Tahun Terbit	2024
Nama Penulis	T. Pöllabauer, A. Pramod, V. Knauthe, M. Wahl
Jurnal	STAG: Smart Tools and Applications in Graphics (Conference)
Kesimpulan	Kami mengeksplorasi teknik akselerasi untuk estimator pose 6D GDRNPP. Surface region attention meningkatkan performa tetapi tidak mempercepat inferensi, sementara penggunaan <i>backbone</i> yang lebih kecil meningkatkan kecepatan dengan kehilangan performa yang minimal. Knowledge distillation memberikan sedikit peningkatan tetapi memerlukan pelatihan dua model terpisah. saya menemukan bahwa geometric head memiliki dampak lebih besar terhadap waktu inferensi dibandingkan modul Patch PnP, sehingga saya mengurangi jumlah parameternya melalui pruning. saya mengembangkan konfigurasi yang menyeimbangkan akurasi dan kecepatan, terutama konfigurasi P28, yang mengalami penurunan performa

	sebesar 4.7%, tetapi meningkatkan latensi hingga 82%. Temuan saya membawa estimasi pose 6D <i>state-of-the-art</i> selangkah lebih dekat menuju inferensi <i>real-time</i>.
Persamaan	Sama-sama diarahkan untuk aplikasi 6D pose estimation dalam konteks real-time, yang krusial untuk robotika dan industri. Fokus pada optimasi efisiensi (speed/latency) dari model yang ada.
Perbedaan	Tidak menggunakan kamera stereoskopik

Penelitian Terdahulu 5	
Judul	StereoPose: Category-Level 6D Transparent Object Pose Estimation from Stereo Images via Back-View NOCS
Tahun Terbit	2024
Nama Penulis	Chen Kai, James Stephen, Congying Sui, Yun-Hui Liu, Pieter Abbeel, dan Qi Dou
Jurnal	(Misalnya, CVPR 2024)

Kesimpulan	Dalam penelitian ini, saya mengusulkan StereoPose, sebuah kerangka kerja stereo image baru untuk estimasi pose objek transparan tingkat kategori. StereoPose membagi estimasi pose tingkat kategori menjadi tiga tahap: estimasi ukuran objek, estimasi awal pose, dan penyempurnaan pose, memungkinkan estimasi pose yang lebih akurat dan tangguh hanya dengan menggunakan citra stereo. saya memperkenalkan back-view NOCS map untuk objek transparan, yang menggali informasi struktur di bagian belakang objek guna meningkatkan akurasi estimasi pose. Selain itu, saya mengusulkan parallax attention module untuk meningkatkan fusi fitur stereo. saya juga merancang epipolar geometry loss guna meningkatkan konsistensi stereo-view NOCS maps. Eksperimen ekstensif pada dataset TOD menunjukkan keunggulan StereoPose dalam estimasi pose 6D objek transparan tingkat kategori.
Persamaan	Sama-sama menggunakan data stereo sebagai input untuk

	meningkatkan akurasi estimasi pose.
Perbedaan	Dirancang khusus untuk objek transparan dan berbasis kategori-level, dengan modul tambahan untuk menangani transparansi dan aliasing yang tidak ada dalam metode single-object stereoskopik.

BAB 3

ANALISIS DAN PERANCANGAN

3.1 Rekayasa Perencanaan

Rekayasa kebutuhan dalam penelitian ini terbagi menjadi dua kategori, yaitu kebutuhan fungsional dan kebutuhan non-fungsional. Perencanaan ini bertujuan memastikan sistem estimasi 6D pose dapat berfungsi sesuai dengan target yang diharapkan.

3.1.1 Kebutuhan Fungsional

Kebutuhan fungsional mencakup fitur utama yang harus dimiliki sistem agar dapat beroperasi dengan baik. Dalam penelitian ini, kebutuhan fungsional meliputi:

- A. Sistem harus mampu menerima input berupa citra stereo dari kamera dan memprosesnya menjadi pasangan gambar kiri dan kanan.
- B. Sistem harus melakukan kalibrasi kamera untuk mengoreksi distorsi lensa serta memastikan akurasi perhitungan kedalaman.
- C. Sistem harus melakukan preprocessing, termasuk koreksi distorsi, normalisasi, dan perhitungan disparitas untuk memperoleh informasi kedalaman objek.
- D. Sistem harus menerapkan model deep learning yang telah dimodifikasi untuk melakukan estimasi 6D pose objek berdasarkan citra stereo.

- E. Sistem harus mampu mengevaluasi hasil estimasi dengan metrik akurasi yang sesuai, seperti error translasi dan rotasi.

3.1.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mencakup aspek kualitas, efisiensi, dan keterbatasan teknis yang perlu dipenuhi agar sistem dapat berjalan optimal. Beberapa kebutuhan non-fungsional yang diidentifikasi adalah sebagai berikut:

- A. Sistem harus kompatibel dengan format foto umum seperti jpg/png serta mampu membaca input dari kamera stereo.
- B. Sistem harus mengelola sumber daya komputasi secara efisien, terutama dalam penggunaan CPU dan GPU, agar pemrosesan berjalan dengan performa optimal.
- C. Sistem harus dapat menangani variasi ukuran dan jumlah dataset tanpa mengalami degradasi kinerja yang signifikan.
- D. Sistem harus memiliki arsitektur yang modular dan fleksibel, sehingga dapat dikembangkan lebih lanjut untuk berbagai skenario aplikasi.

3.2 Pemilihan Model Dan Tahapan Pemecah Masalah

Penelitian ini bertujuan untuk melakukan estimasi pose 6D dari objek tunggal menggunakan aliran video stereoskopik secara langsung. Untuk mencapai tujuan tersebut, pemilihan metode yang tepat sangat krusial guna memastikan akurasi dan efisiensi dalam pemrosesan data stereoskopik. Beberapa pertimbangan pemilihan metode dan algoritma yang digunakan dalam penelitian ini dijelaskan sebagai berikut.

3.2.1 Koreksi Distorsi Lensa Dan Kalibrasi Kamera

Pemilihan metode koreksi distorsi dan kalibrasi kamera dilakukan untuk memastikan bahwa citra yang dihasilkan dari kamera stereo sesuai dengan format yang dapat diolah lebih lanjut dalam model estimasi pose. Beberapa alasan pemilihan metode ini adalah:

- A. Menghilangkan distorsi lensa Kamera yang digunakan memiliki efek fisheye yang dapat menyebabkan kesalahan dalam estimasi kedalaman. Koreksi distorsi dilakukan menggunakan algoritma berbasis model pinhole dengan parameter intrinsik yang diperoleh dari proses kalibrasi.
- B. Kalibrasi kamera stereo, Kalibrasi dilakukan untuk menentukan parameter intrinsik dan ekstrinsik dari kedua kamera sehingga dapat digunakan dalam perhitungan disparitas dan rekonstruksi kedalaman.
- C. Peningkatan akurasi pose estimation, Dengan data yang telah dikoreksi, model estimasi pose dapat bekerja lebih akurat dalam mengekstraksi fitur dan menghitung transformasi objek dalam ruang 3D.

3.2.2 Pemilihan Versi Model ConvNeXt

Untuk menentukan backbone yang paling cocok dalam tugas estimasi pose 6D berbasis stereo, diambil perbandingan beberapa versi model ConvNeXt dan ConvNeXt V2 dari Pöllabauer, et al; Pemilihan didasarkan pada keseimbangan

antara akurasi, efisiensi komputasi, dan waktu inferensi[16]. Tabel berikut menunjukkan ringkasan performa masing-masing model.

Tabel 3.2.2: Perbandingan ConvNext

Model	GFLOP	Throughput (img/s)	Top-1 Acc (%)	AR (TD)	AR (DD)	Latency (ms)
ConvNeXt-B	15.4	1,485	83.8	68.6	69.0	–
ConvNeXt-S	8.7	2,008	83.1	68.2	68.7	–
ConvNeXt-V2-T	4.47	1,452.7	83.8	68.0	68.0	–
ConvNeXt-V2-N	2.45	2,300.2	82.1	67.3	66.0	49.2
ConvNeXt-V2-P	1.37	3,274	80.3	65.0	–	43.79

ConvNeXt V2 NANO dipilih sebagai arsitektur backbone dalam penelitian ini karena ia menawarkan keseimbangan optimal antara akurasi, efisiensi komputasi, dan kecepatan inferensi; tiga aspek krusial dalam tugas estimasi pose 6D berbasis stereo. Meskipun memiliki jumlah parameter yang relatif kecil (15.6 juta) dan komputasi rendah (2.45 GFLOPs), model ini tetap mampu mencapai akurasi top-1 sebesar 82.1% pada klasifikasi ImageNet, yang setara dengan arsitektur yang lebih besar.

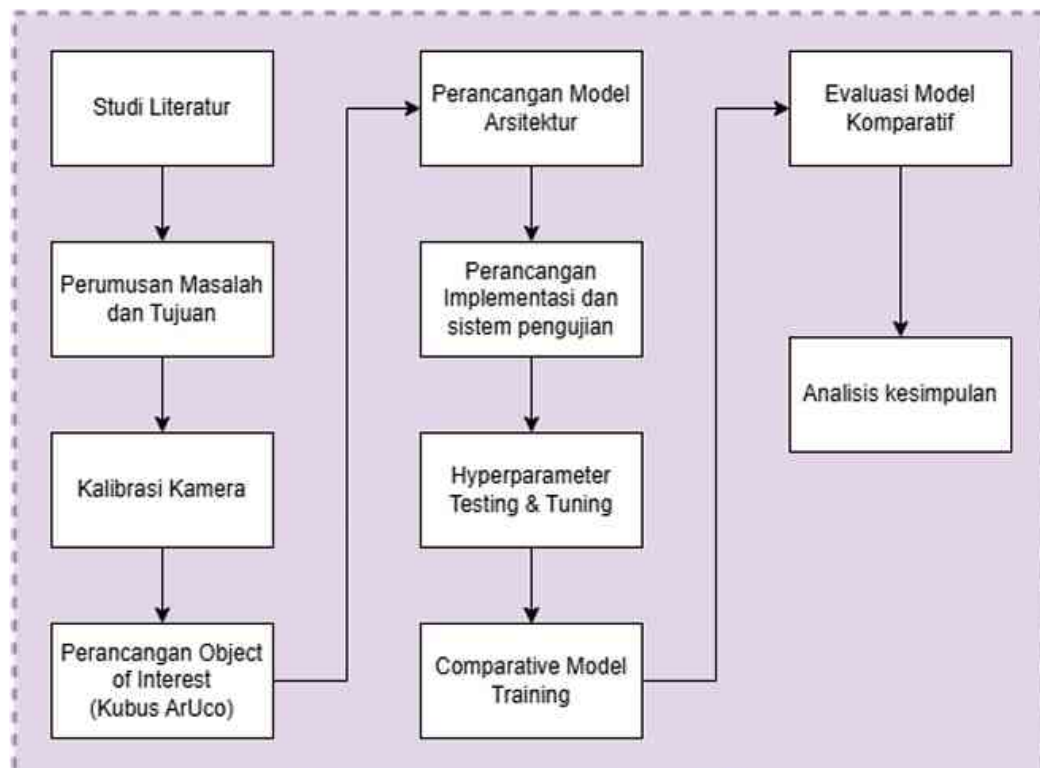
Lebih penting lagi, ConvNeXt V2 NANO memiliki throughput tinggi sebesar 2,300 gambar per detik dan latency inferensi yang sangat kompetitif

(sekitar 49.2 ms), membuatnya sangat cocok untuk aplikasi real-time seperti pengolahan citra stereo dalam estimasi pose 6D. Pöllabauer, et al., menemukan pada beberapa dataset pose 6D, konfigurasi berbasis ConvNeXt-V2-N juga mencatat nilai Average Recall (AR) yang tinggi (67.3% tanpa distilasi), menjadikannya pilihan kuat dibandingkan arsitektur lain yang lebih kompleks tetapi lebih lambat atau lebih mahal secara komputasi[16].

Kombinasi antara akurasi yang baik, efisiensi, dan waktu inferensi yang rendah membuat ConvNeXt V2 NANO menjadi pilihan strategis dalam penelitian ini, terutama dalam konteks keterbatasan perangkat keras (GPU RTX 3070 8GB) dan kebutuhan performa real-time.

3.3 Perancangan Proses

Perancangan proses dalam penelitian ini disusun secara sistematis untuk memastikan bahwa analisis komparatif terhadap arsitektur fusi stereo dapat berjalan secara valid dan terukur. Alur penelitian mencakup beberapa tahapan kunci, mulai dari persiapan dan pengolahan data mentah, pembangunan dan pelatihan model, hingga evaluasi kuantitatif untuk menarik kesimpulan. Setiap tahap dirancang untuk mengisolasi variabel-variabel tertentu sehingga perbandingan antar arsitektur dapat dilakukan secara adil.

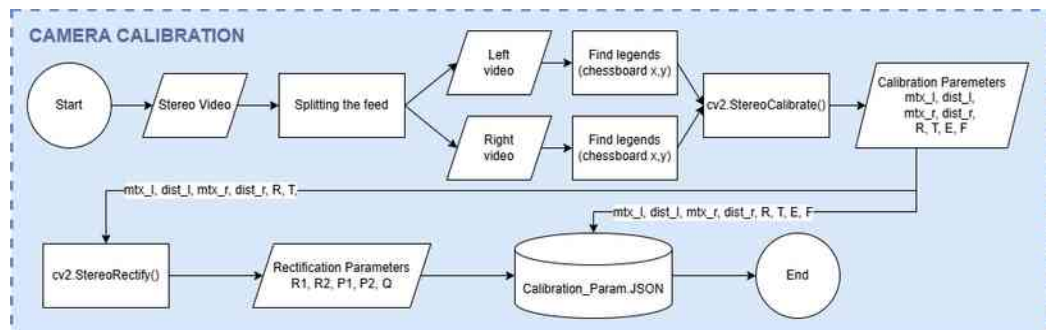


Gambar 3.1: Metode Riset

Alur Metode Penelitian yang diterapkan dalam penelitian ini diawali dengan tahap Studi Literatur untuk mengkaji penelitian terdahulu dan mengidentifikasi celah pengetahuan dalam arsitektur fusi stereo. Berdasarkan temuan tersebut, dilakukan Perumusan Tujuan Penelitian yang spesifik untuk membandingkan efektivitas berbagai strategi fusi. Tahap selanjutnya adalah Perancangan Model Arsitektur, di mana tiga pendekatan fusi yang berbeda (Baseline, SNN, dan Early Fusion) dirancang di atas backbone ConvNeXt V2 NANO yang konsisten. Sebelum perbandingan utama, dilakukan serangkaian Pengujian dan Penyetelan Hyperparameter secara empiris untuk menentukan konfigurasi yang paling adil dan seimbang bagi ketiga model. Dengan parameter

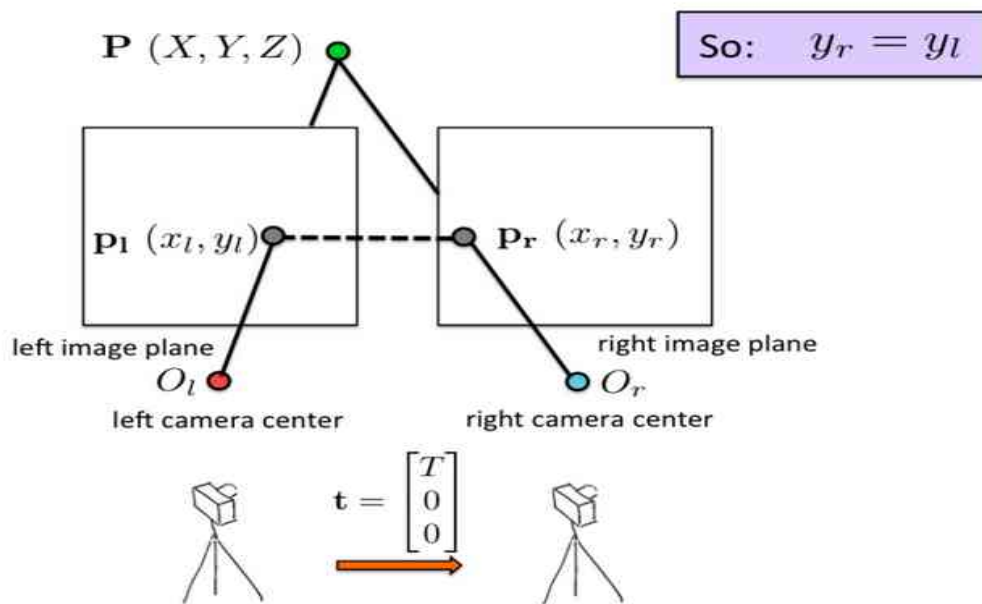
yang telah terstandarisasi, ketiga model dilatih dalam tahap Pelatihan Model Komparatif. Kinerja masing-masing model kemudian diukur secara kuantitatif dalam tahap Evaluasi Komparatif Model menggunakan metrik RMSE, kecepatan inferensi, dan penggunaan VRAM. Hasil dari model-model terbaik selanjutnya diintegrasikan ke dalam sebuah dasbor Streamlit pada tahap Implementasi dan Pengujian Sistem untuk validasi kualitatif dan demonstrasi. Terakhir, seluruh data kuantitatif dan kualitatif yang terkumpul dianalisis secara mendalam pada tahap Analisis Kesimpulan untuk menjawab rumusan masalah dan menarik kesimpulan akhir mengenai arsitektur fusi stereo yang paling optimal.

3.3.1 Kalibrasi Kamera



Gambar 3.2: Kalibrasi Kamera

Abdelaal, et al., menjelaskan bahwa alibrasi kamera stereo merupakan proses penting untuk memperoleh parameter intrinsik dan ekstrinsik dua kamera[3]. Tahap ini diperlukan agar rekonstruksi spasial dan estimasi kedalaman dapat dilakukan secara akurat.



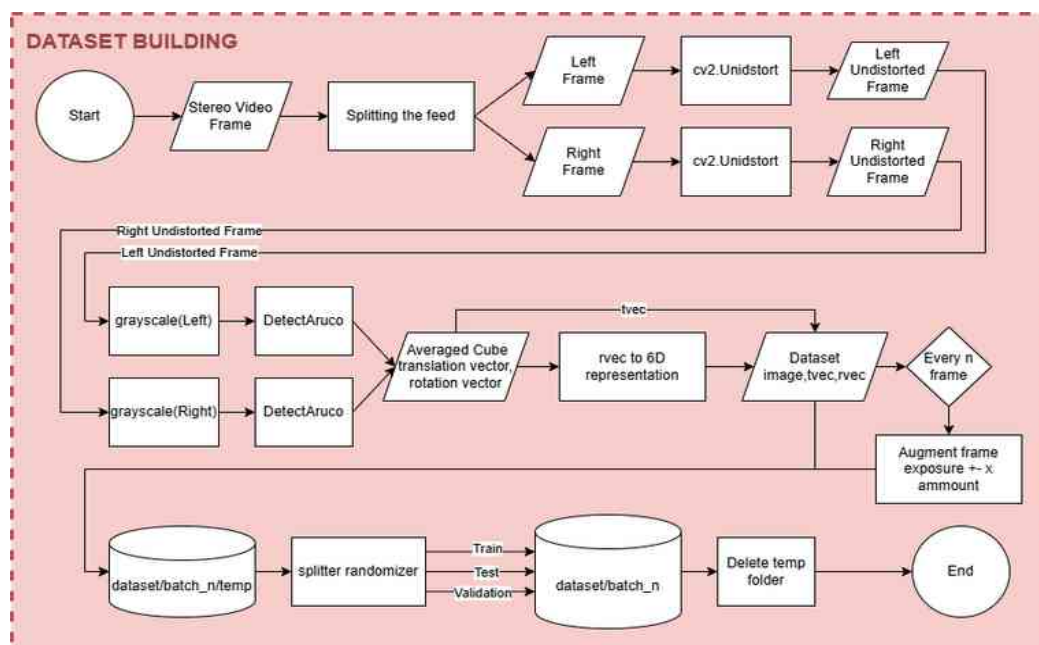
Gambar 3.3: Rektifikasi Stereo

Gambar 3.3. menggambarkan hubungan epipolar antara titik dunia $P(X,Y,Z)$ dan proyeksinya pada bidang gambar kiri dan kanan, yaitu $P_l(x_l,y_l)$ dan $P_r(x_r,y_r)$. Karena sistem kamera telah di-rektifikasi, nilai koordinat vertikal akan sama. Hal ini menyederhanakan proses pencocokan stereo dan memungkinkan perhitungan kedalaman (depth) melalui triangulasi.

Seperti yang dijelaskan oleh Guan, et al., rektifikasi ini memastikan bahwa korespondensi piksel antara kedua citra hanya terjadi sepanjang garis horizontal yang sama, yang menjadi dasar perhitungan kedalaman dalam pengolahan citra stereo[7].

3.3.2 Akuisisi Data Dan Labeling

Dataset stereo dibangun dari pasangan citra kiri dan kanan, yang dikaitkan dengan anotasi ground truth untuk keperluan pelatihan model estimasi pose. Penggunaan data yang dihasilkan secara terkontrol seperti ini merupakan pendekatan yang valid dan umum digunakan, terutama untuk melatih model agar dapat mengenali pose objek dengan akurat tanpa memerlukan dataset dunia nyata yang besar dan sulit dianotasi [17].



Gambar 3.4: Akuisisi data and Labeling

Setiap pasangan citra diproses melalui tahap preprocessing, seperti normalisasi, pengubahan color space, dan augmentasi data. Data disusun dalam struktur direktori yang memisahkan pelatihan, validasi, dan pengujian. Selain itu, ground truth pose dalam bentuk rotasi kontinu 6D Zhou dan translasi dalam satuan 3D Euler juga disertakan. Format dataset akan disesuaikan agar kompatibel dengan berbagai skema arsitektur model[18]; Termasuk model Twin head Siamese Neural Network dan model dengan masukan Early fusion 6 Channel.

Dataset yang digunakan dalam penelitian ini disusun untuk melatih dan mengevaluasi model estimasi pose 6D. Setiap sampel data terdiri dari dua komponen utama: citra input dan label ground truth. Pendekatan ini memastikan bahwa model dapat belajar memetakan informasi visual dari citra ke koordinat spasial tiga dimensi.

Input utama untuk setiap arsitektur model adalah pasangan citra stereoskopik yang telah melalui proses rektifikasi dan koreksi distorsi. Setiap pasangan citra ini, yang diambil dari frame video, berfungsi sebagai representasi visual dari objek pada satu titik waktu. Citra-citra ini kemudian diolah sesuai dengan kebutuhan masing-masing arsitektur, baik digabungkan menjadi satu gambar, diproses secara terpisah, maupun ditumpuk menjadi input 6-channel.



Gambar 3.5: Citra Pada Dataset

Setiap pasangan citra input diasosiasikan dengan sebuah label ground truth yang disimpan dalam berkas labels.csv. Label ini terdiri dari 9 nilai numerik yang mendefinisikan pose 6D objek secara lengkap dan terbagi menjadi dua bagian. Tiga nilai pertama (x , y , z) merepresentasikan vektor translasi, yang menunjukkan posisi pusat objek dalam ruang tiga dimensi relatif terhadap kamera. Enam nilai berikutnya ($r1x$, $r1y$, $r1z$, $r2x$, $r2y$, $r2z$) merepresentasikan orientasi objek menggunakan representasi rotasi kontinu 6D. Menurut Zhou et al., representasi ini dipilih karena kemampuannya untuk menghindari diskontinuitas dan masalah gimbal lock yang sering terjadi pada representasi lain seperti sudut Euler, sehingga proses pembelajaran menjadi lebih stabil untuk jaringan saraf [9]. Seluruh dataset ini kemudian dibagi menjadi tiga set data terpisah: data pelatihan (training), data validasi (validation), dan data pengujian (testing), untuk memastikan evaluasi model yang objektif.

3.3.3 Pembangunan Model

Dalam penelitian ini, tiga arsitektur model dipilih untuk dibandingkan dalam konteks estimasi pose 6D berbasis citra stereo, yaitu model baseline dengan

input RGB hasil resize sesuai IMG_SIZE dari stereo image feed, model Siamese Neural Network dengan Shared Weights, dan model Early Fusion, karena banyak pendekatan CNN konvensional yang belum sepenuhnya memanfaatkan fitur-fitur unik dari data stereo seperti disparitas perspektif dan tekstur secara optimal. Pemilihan model baseline bertujuan sebagai tolak ukur sederhana; Kritik Navarro dan Loaiza, menyebutkan bahwa banyak pendekatan CNN tidak memanfaatkan kedalaman stereo secara optimal[19].

Sementara itu, arsitektur Siamese digunakan karena kemampuannya mengekstraksi fitur korespondensi antara dua pandangan dengan efisien melalui bobot bersama, sebagaimana ditunjukkan oleh An dan Yan dalam konteks pelacakan multi-object, serta oleh Jordan & Shridhar dalam estimasi kedalaman stereo[12], [13]. Sebaliknya, model Early Fusion dipilih karena memungkinkan jaringan mengakses dan menggabungkan informasi dari kedua citra sejak tahap awal, sesuai dengan temuan Xu, et al., [6]. Yang menyoroti pentingnya pemrosesan multi-channel untuk tugas-tugas persepsi spasial tingkat lanjut; Seperti pada riset yang dilakukan Navarro dan Loaiza model yang menggunakan 6 Channel memiliki performa yang cukup menarik. Ketiga pendekatan ini dipertimbangkan untuk mengkaji peran dan kontribusi masing-masing dalam meningkatkan akurasi dan efisiensi estimasi pose 6D dari input stereo.

Karena secara harfiah kamera stereo mengambil citra dari dua kamera berbeda pada posisi yang berbeda data dari kedua kamera dapat diasumsikan sebagai gambar multispektral. Citra multispektral adalah sebuah citra yang terdiri

dari beberapa channel yang berbeda[20]. Oleh karena itu, citra multispektral membutuhkan tahapan preprocessing yang berbeda dengan citra pada umumnya.

3.3.3.1 Tanpa Fusi (*Baseline*)

Model dasar hanya menggunakan satu citra gabungan kiri kanan yang ditransformasi menjadi gambar 224x224 untuk memperkirakan pose 6D. Pendekatan ini berfungsi sebagai baseline untuk membandingkan performa dengan varian yang lain.



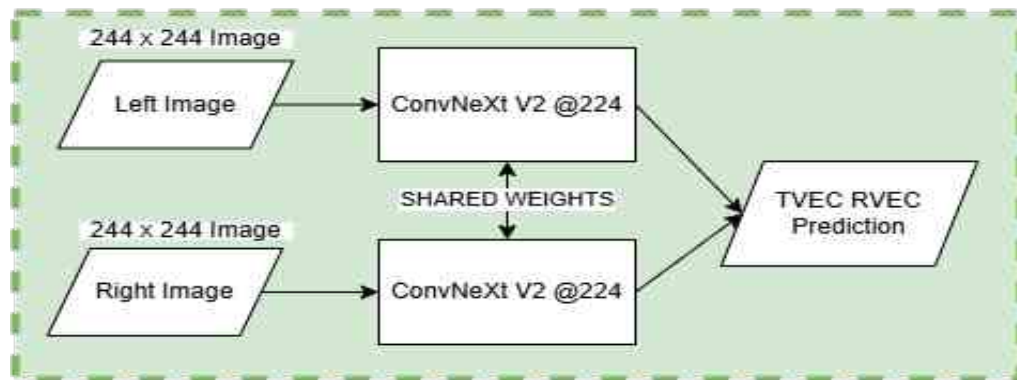
Gambar 3.6: Arsitektur Tanpa Fusi (Baseline)

Citra kiri dan kanan secara tunggal diproses melalui Backbone, kemudian dilemparkan kepada custom regressor head. Arsitektur ini terdiri atas backbone ekstraksi fitur visual, diikuti oleh regresor pose yang memproyeksikan fitur menjadi rotasi dan translasi. Fungsi loss menggabungkan kesalahan translasi (dalam satuan meter) dan kesalahan rotasi (berbasis geodesic loss).

3.3.3.2 Jaringan Saraf Siamese (*Siamese Neural Network*)

Model Siamese Neural Network - Shared Weight memproses citra kiri dan kanan melalui dua jalur identik yang berbagi parameter. Pendekatan ini

mengintegrasikan informasi dari kedua sudut pandang secara simetris tanpa menggandakan parameter yang digunakan.



Gambar 3.7: Arsitektur Jaringan Saraf Siamese
(*Siamese Neural Network*)

Masing-masing citra stereo dilewatkan ke backbone berbeda namun dengan parameter bersama. Fitur dari kedua cabang diekstrak secara paralel lalu concatenate sebelum dimasukkan ke regresor pose. Jalur ini memungkinkan model untuk memahami hubungan spasial antar pandangan stereo secara eksplisit sambil mempertahankan efisiensi parameter. Dibandingkan dengan Baseline, arsitektur ini memanfaatkan dua input visual secara langsung. Riset ingin mencari tahu apakah akurasi akan pose akan naik oleh karena informasi kedalaman yang lebih baik dari masukan stereo, sesuai dengan kesimpulan An dan Yan[13].

3.3.3.3 Fusi Dini (*Early Fusion*)

Model ini menggabungkan langsung citra kiri dan kanan pada channel RGB, menghasilkan input tunggal dengan 6 channel (3 ch kiri + 3 ch kanan); Membuat suatu citra yang memiliki channel warna lR,lG,lB,rR,rG,rB.

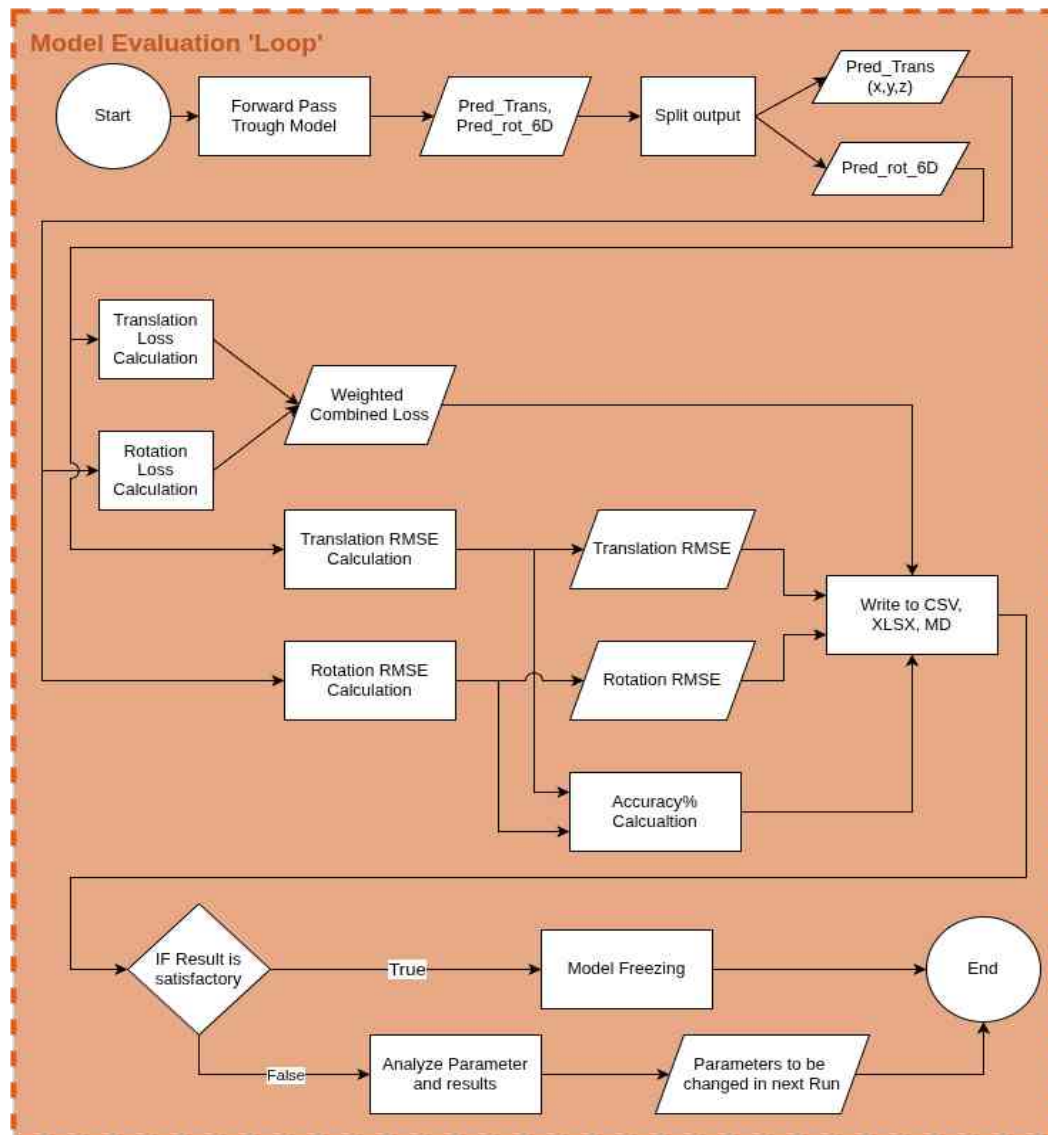


Gambar 3.8: Arsitektur Fusi Dini(Early Fusion)

Input 6 ch dimasukkan ke dalam Backbone yang dimodifikasi pada layer awal untuk menerima input non-standar. Seluruh arsitektur setelahnya tetap serupa dengan model Baseline. Pendekatan ini sangat sederhana dari segi implementasi karena hanya membutuhkan satu jalur pemrosesan dan satu backbone. Namun, karena informasi kiri dan kanan digabungkan secara awal (early fusion), model harus belajar secara implisit bagaimana mengekstraksi kedalaman dan korelasi antar perspektif. Tantangan utama dari metode ini adalah menjaga keseimbangan kontribusi antar pandangan serta memastikan bahwa layer awal cukup kuat untuk memisahkan informasi spasial dari dua sumber. Dalam riset Alaba, et al., early fusion/data fusion ditemukan memiliki performa yang tidak kalah jauh dengan teknik fusi informasi lainnya seperti middle fusion atau late fusion(SNN)[4].

3.3.4 Model Evaluation

Evaluasi dilakukan terhadap model yang telah dilatih menggunakan dataset test dan val untuk mengukur akurasi estimasi pose serta efisiensi.



Gambar 3.9: Metode Evaluasi Model

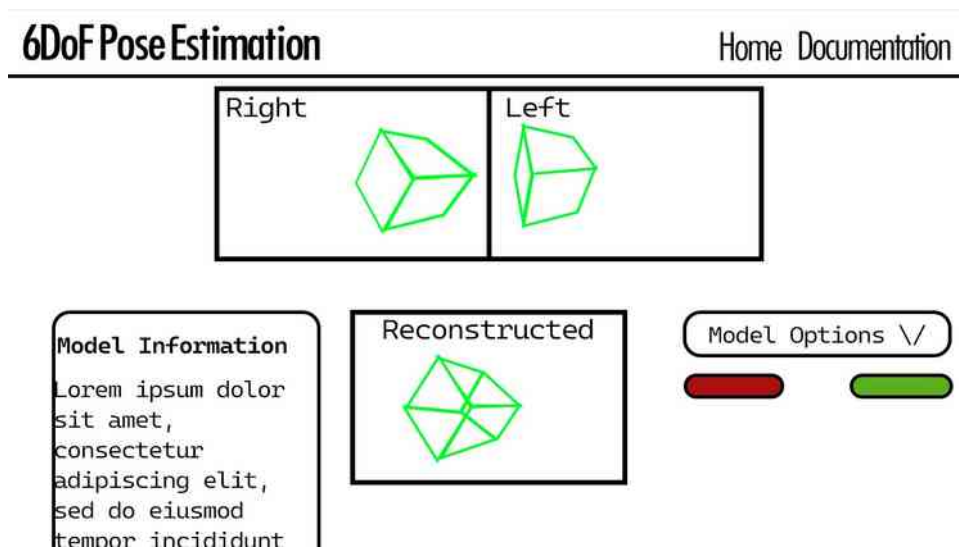
Setiap model yang telah dilatih kemudian dievaluasi secara sistematis menggunakan satu loop evaluasi terpadu yang dirancang untuk memastikan perbandingan yang adil dan dapat direplikasi. Proses ini dimulai dengan memuat set data pengujian (testing) dan validasi (validation) yang telah dipisahkan. Selama inferensi, teknik Automatic Mixed Precision (AMP) diimplementasikan

untuk mengoptimalkan penggunaan memori dan mempercepat kalkulasi pada GPU. Untuk setiap batch data, beberapa metrik kinerja utama dihitung, mencakup Loss (kesalahan gabungan), RMSE translasi (dalam sentimeter), dan RMSE rotasi (dalam derajat). Seluruh hasil, termasuk waktu inferensi per gambar, dicatat secara otomatis ke dalam tiga format berbeda untuk analisis yang komprehensif: berkas .xlsx untuk data numerik terstruktur, berkas .md untuk laporan kualitatif per model, dan log TensorBoard untuk visualisasi kurva pelatihan dan validasi. Pendekatan evaluasi yang seragam ini diterapkan pada semua varian arsitektur untuk menjamin bahwa perbandingan kinerja yang dihasilkan murni merefleksikan perbedaan antara pendekatan fusi stereo, dalam skenario dan kondisi yang sepenuhnya setara.

3.4 Perancangan Tampilan Aplikasi

3.4.1 *Home Page*

Antarmuka pengguna dibangun menggunakan framework Streamlit untuk menyajikan visualisasi dan interaksi hasil estimasi pose 6 DoF secara real-time. Halaman utama dirancang agar informatif namun tetap sederhana untuk kemudahan penggunaan.



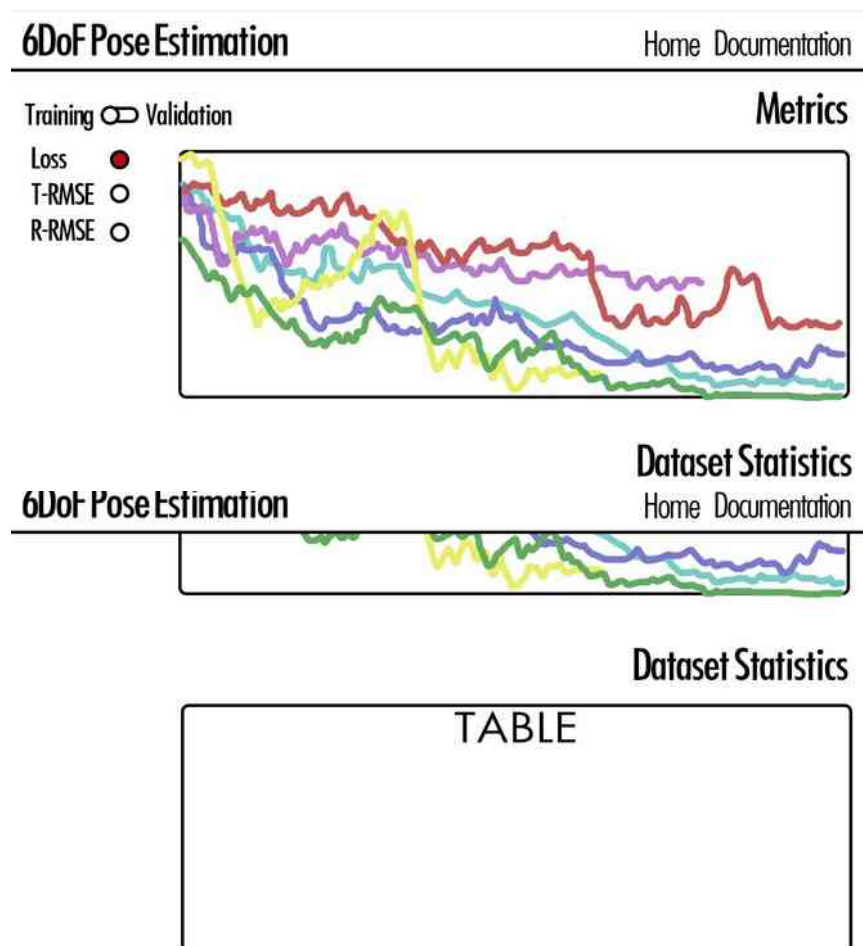
Gambar 3.10: Streamlit Home Page GUI

Tampilan utama terdiri atas tiga komponen visual utama: proyeksi objek dari kamera kiri dan kanan (Left dan Right), serta hasil rekonstruksi pose 3D (Reconstructed). Kedua gambar input ditampilkan secara berdampingan untuk memudahkan verifikasi visual terhadap hasil prediksi jaringan. Di bagian bawah, hasil rekonstruksi dari estimasi pose digambarkan secara terpisah dengan posisi dan orientasi objek 3D yang diperoleh dari model prediktif.

Komponen Model Information di sisi kiri menyediakan informasi ringkas mengenai arsitektur dan konfigurasi model yang sedang digunakan. Pengguna dapat memilih model yang tersedia melalui menu dropdown Model Options, dan melakukan evaluasi langsung dengan indikator visual berupa tombol aktif berwarna hijau (berhasil) atau merah (gagal) yang menggambarkan status inferensi atau validasi. Desain antarmuka ini mempermudah eksplorasi hasil eksperimen dan validasi model estimasi pose dalam skenario stereo vision, serta mendukung proses analisis kualitatif dan debugging secara efisien.

3.4.2 Documentation

Halaman dokumentasi dirancang untuk memvisualisasikan performa pelatihan dan validasi dari model estimasi pose 6DoF secara interaktif. Antarmuka ini bertujuan memberikan pemahaman menyeluruh terhadap konvergensi model melalui analisis metrik selama proses training.



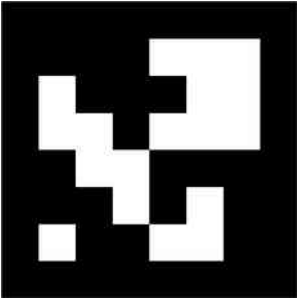
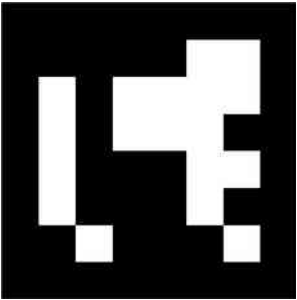
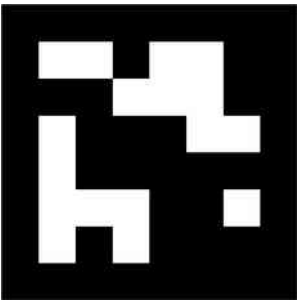
Gambar 3.11: Streamlit Home Page GUI

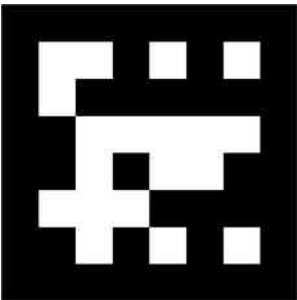
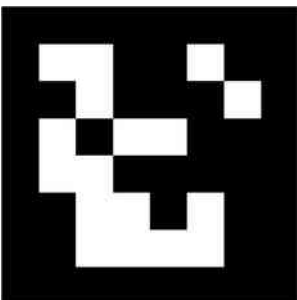
Pada bagian utama, ditampilkan grafik tren metrik pelatihan seperti Loss, Translational RMSE (T-RMSE), dan Rotational RMSE (R-RMSE) untuk masing-masing sesi pelatihan dan validasi. Pengguna dapat memilih metrik yang ingin ditampilkan melalui tombol kontrol interaktif di sisi kiri grafik, serta beralih antara kurva pelatihan dan validasi menggunakan toggle switch di bagian atas. Di bagian kanan atas halaman terdapat judul Metrics, sedangkan bagian kanan bawah menyajikan tautan ke halaman Dataset Statistics yang menyediakan informasi kuantitatif dan distribusi data latih. Desain halaman dokumentasi ini disusun untuk mendukung proses evaluasi eksperimen secara transparan dan membantu dalam proses tuning hyperparameter serta pemilihan model terbaik berdasarkan performa empiris.

3.5 Perancangan Tracking Object

Untuk menghasilkan data ground truth yang konsisten dan akurat, diperlukan sebuah objek fisik yang posenya dapat dilacak secara andal dari berbagai sudut pandang. Dalam penelitian ini, objek yang dirancang adalah sebuah kubus dengan marker ArUco yang unik ditempelkan pada setiap sisinya. Penggunaan marker yang berbeda untuk setiap sisi memungkinkan sistem estimasi pose untuk tidak hanya mendeteksi objek, tetapi juga mengidentifikasi orientasi spesifiknya (sisi mana yang menghadap kamera) [21]. Konfigurasi ini sangat krusial untuk menghasilkan label rotasi 6D yang presisi selama proses pembuatan dataset. Tabel berikut merinci pemetaan antara setiap sisi kubus dengan ID marker ArUco dari kamus DICT_6X6_250 yang digunakan.

Tabel 3.1: ArUco Marker Kubus

<i>FACES</i>	<i>ARUCO MARKERS</i>
<i>Front</i>	
<i>Back</i>	
<i>Right</i>	

<i>Left</i>	
<i>Top</i>	
<i>Bottom</i>	

3.6 Jadwal Pengerjaan

Jadwal pengerjaan skripsi akan berlaku sebagai berikut:

Tabel 3.2: Jadwal Pengerjaan

Tahap	Bulan 1	Bulan 2	Bulan 3	Bulan 4	Bulan 5	Bulan 6
Proposal						
Evaluasi feasibilitas						
Akuisisi dataset						
Model training & tuning						
Pengembangan sistem						
Penyelesaian Laporan						

BAB 4

IMPLEMENTASI

4.1 Implementasi Penelitian

Gambar berikut menampilkan implementasi model 6D Pose Estimation menggunakan antarmuka web yang dikembangkan dengan **Streamlit**. Antarmuka ini memungkinkan pengguna untuk memilih sumber input, memuat model, dan menampilkan hasil estimasi pose secara interaktif.

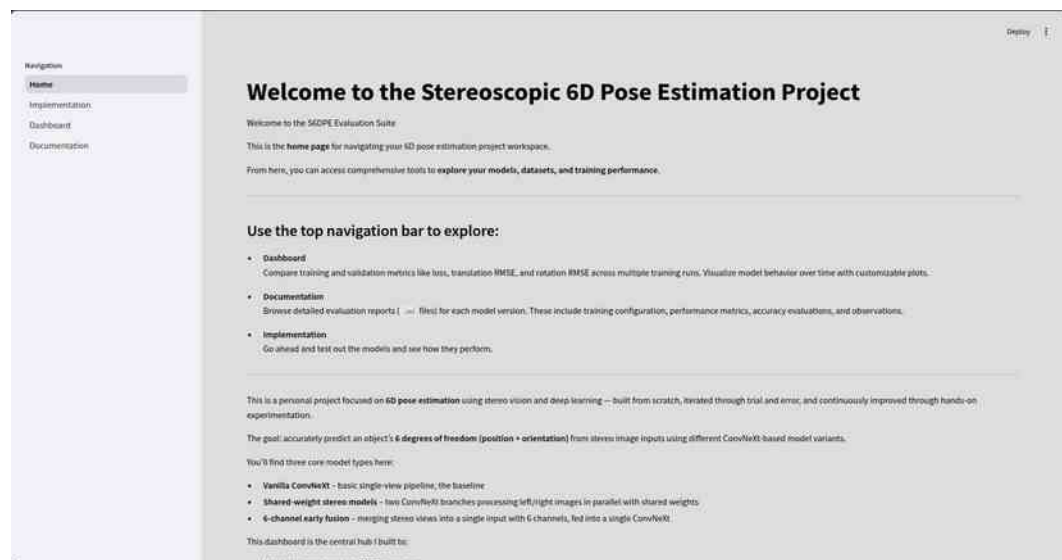


Gambar 4.1: Implementasi Streamlit

Tangkapan layar ini menunjukkan proses kerja sistem 6D Pose Estimation yang memanfaatkan model deep learning dalam antarmuka berbasis web (WebGUI). Pengguna dapat memilih sumber input berupa webcam stereo, file

video yang diunggah, video lokal, atau gambar statis. Setelah input dipilih, pengguna dapat memilih model terlatih (misalnya CLEAN-S-ConvNeXt6DP5.3.pt) untuk melakukan inferensi pada data input. Hasil prediksi, berupa translasi 3D dan rotasi 6D objek, ditampilkan dalam antarmuka bersama dengan input stereo yang digunakan. Proses ini mencakup pengolahan data stereo, pemrosesan model, dan visualisasi hasil estimasi pose, sehingga sistem ini mendemonstrasikan alur kerja yang terintegrasi dalam konteks aplikasi 6D Pose Estimation berbasis deep learning.

Halaman berikut memperlihatkan tampilan awal Home pada aplikasi dashboard Streamlit untuk penelitian estimasi pose 6D berbasis stereo vision.

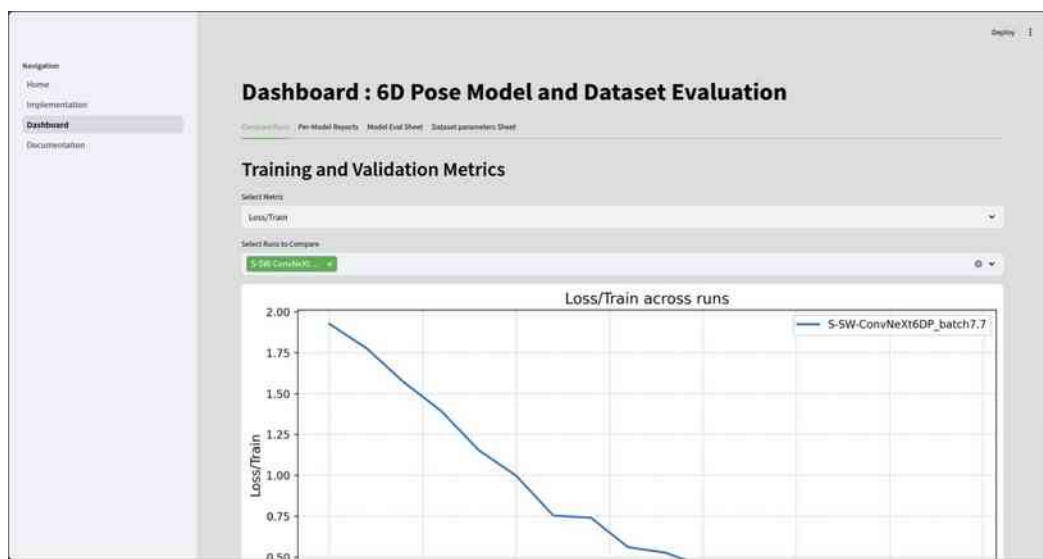


Gambar 4.2: Implementasi HomePage

Pada halaman Home, pengguna disambut dengan judul penelitian dan penjelasan singkat mengenai tujuan utama dashboard yaitu sebagai pusat evaluasi model, dataset, serta performa training. Halaman ini juga memberikan navigasi

singkat ke berbagai fitur seperti Dashboard, Documentation, dan Implementation, serta ringkasan tentang pendekatan model yang tersedia, sehingga user dapat memahami alur pemanfaatan aplikasi sejak awal.

Tampilan berikut menunjukkan submodul Dashboard untuk analisis Training and Validation Metrics.



Gambar 4.3: Implementasi Dashboard

Pada bagian ini, pengguna dapat membandingkan metrik pelatihan lintas beberapa training run (seperti Loss/Train, Loss/Val, Translation RMSE, Rotation RMSE) secara visual dalam bentuk grafik. Pengguna dapat memilih metrik yang ingin dianalisis serta memilih satu atau beberapa sesi pelatihan untuk dibandingkan. Fitur ini mempermudah identifikasi tren konvergensi dan evaluasi apakah model mengalami overfitting/underfitting selama training.

Tampilan berikut merupakan tampilan Dataset Parameters Sheet pada dashboard evaluasi.

Dashboard : 6D Pose Model and Dataset Evaluation

Compare Runs | Per-Model Reports | Model Eval Sheet | [Dataset Parameters Sheet](#)

Dataset Parameters Sheet (dataset_info.csv)

timestamp	batch_id	total_frames	frames_split	train_aug	train_pose	val_pose	test_pose	w_val_test_ratio	pose_method	pose_normalization
2025-05-11 13:33:33	1	2383	3	0	1396	233	457	75.05:20	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-12 13:23:37	2	6798	3	8	4759	479	1361	75.05:20	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-13 00:36:54	3	6799	3	8	4079	189	1391	90.20:20	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-13 04:49:34	4	4799	3	8	5439	679	961	88.02:10	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-21 10:26:09	5	21713	3	2	17369	1173	2172	88.02:10	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-24 05:35:56	6	14474	3	1	11579	1447	1448	88.02:10	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation
2025-05-28 05:34:41	7	7237	3	0	5769	723	725	90.02:10	ArUco-based 6DOF Pose Estimation	2D-to-6D Rotational Representation

[Download as CSV](#)

Dataset Metadata (All Batches)

Metadata for dataset batch 1

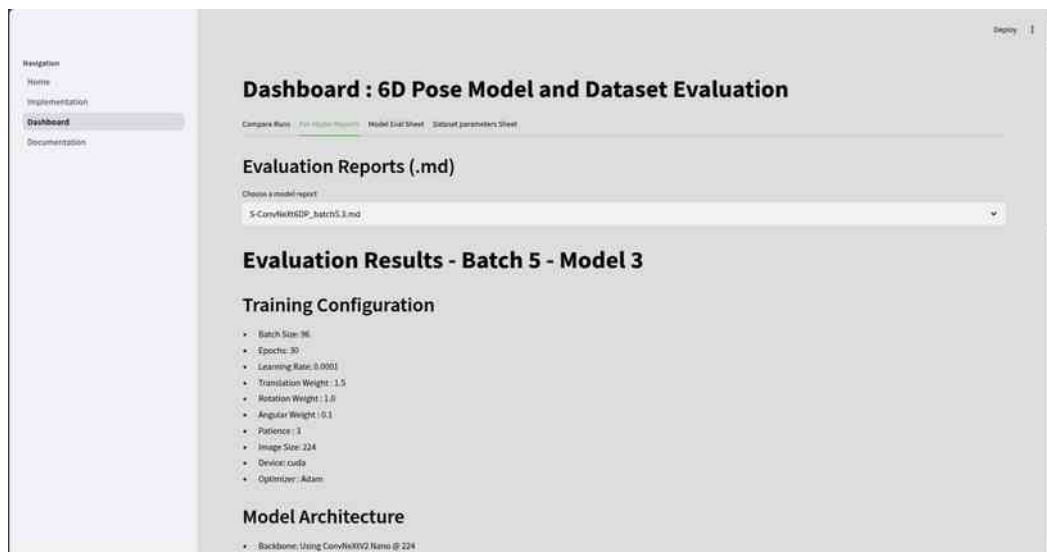
Batch 1 Metadata

Date: 2025-05-17T23:17:38.672963

Gambar 4.4: Implementasi Dashboard Dataset Info

Bagian ini menyediakan tabel terstruktur mengenai parameter dataset yang digunakan dalam setiap batch eksperimen, mencakup informasi seperti jumlah total frame, rasio split dataset (train, val, test), metode pose yang digunakan, serta metode normalisasi rotasi. Pengguna juga dapat mengunduh data ini dalam format CSV. Selain itu, metadata lengkap untuk setiap batch dataset juga dapat diakses, memudahkan tracking sumber data dan replikasi eksperimen.

Halaman berikut adalah tampilan submodul “Per-Model Reports” pada dashboard evaluasi.



Gambar 4.5: Implementasi Dashboard, Dataset MD

Pada panel ini, pengguna dapat memilih dan membaca laporan evaluasi terperinci untuk masing-masing model yang dihasilkan selama eksperimen. Laporan ini umumnya berisi konfigurasi pelatihan, rincian arsitektur, serta penjelasan hasil evaluasi per-model dalam format markdown, sehingga memudahkan proses interpretasi hasil, dokumentasi, serta berbagi temuan antar anggota tim.

Gambar berikut memperlihatkan bagian “Evaluation Summary Table (EVAL.xlsx)” pada dashboard.

Navigation

Home

Implementation

Dashboard

Documentation

Dashboard : 6D Pose Model and Dataset Evaluation

Complete Run

Per Model Reports

Model Eval Table

Dataset parameters View

Evaluation Summary Table (EVAL.xlsx)

ID	IMG_SIZE	IMG_FMT	HEAD_ARCH	PARAM_COUNT	BATCH_SIZE	EPS	RTFL_ID	RTFL_LEN	LR_RT	TRAIN_TIME	VRAM_USED	PAL_SIZE	PAL_SIZE	PAL_TRAIN_SIZE	T1_TRAIN_SIZE	
0	512	384	VANILA	640 + 512 + 9	332288	32	38	7	23711	0.0001	120	8.4	0.0149	0.3388	0.0463	0.0007
1	71	384	VANILA	640 + 512 + 9	332288	32	38	7	7297	0.0001	80	8	0.0002	0.0384	0.0314	0.0408
2	72	328	VANILA	640 + 256 + 9	140000	32	38	7	7137	0.0001	85	8	0.0045	0.0793	0.0339	0.0586
3	73	328	VANILA	640 + 256 + 9	140000	84	28	7	7137	0.0001	80	6	0.7643	0.7799	0.0683	0.0832
4	74	324	VANILA	640 + 256 + 128 + 9	187788	32	38	7	7137	0.0001	7.8	3	1.7640	1.9687	0.075	0.0749
5	75	328	VANILA	640 + 256 + 9	140000	32	38	7	7137	0.0001	29.4	8	0.0007	0.0761	0.039	0.0381
6	81	324	VANILA	640 + 256 + 9	140000	96	36	8	14478	0.0001	65.2	7	0.2601	0.2718	0.0318	0.0328
7	81	328	VANILA	640 + 256 + 9	140000	96	38	8	14478	0.0001	68.2	7	0.2677	0.2763	0.0324	0.0321
8	82	324	6d	640 + 256 + 9	140000	48	23	8	14478	0.0001	89	4	0.201	0.2438	0.0473	0.0453
9	76	324	6d	640 + 256 + 128 + 9	187788	96	23	7	7137	0.0001	37.8	7.3	0.7706	0.7883	0.0389	0.0383

Download as CSV

In the Table there are 2 Variants of model:

•

VANILA: A standard single-image ComNet2 pipeline trained on the left stereo image only. This serves as a baseline to compare stereo-based improvements. Input is a regular 3-channel (RGB) image, and the network predicts 6D pose directly.

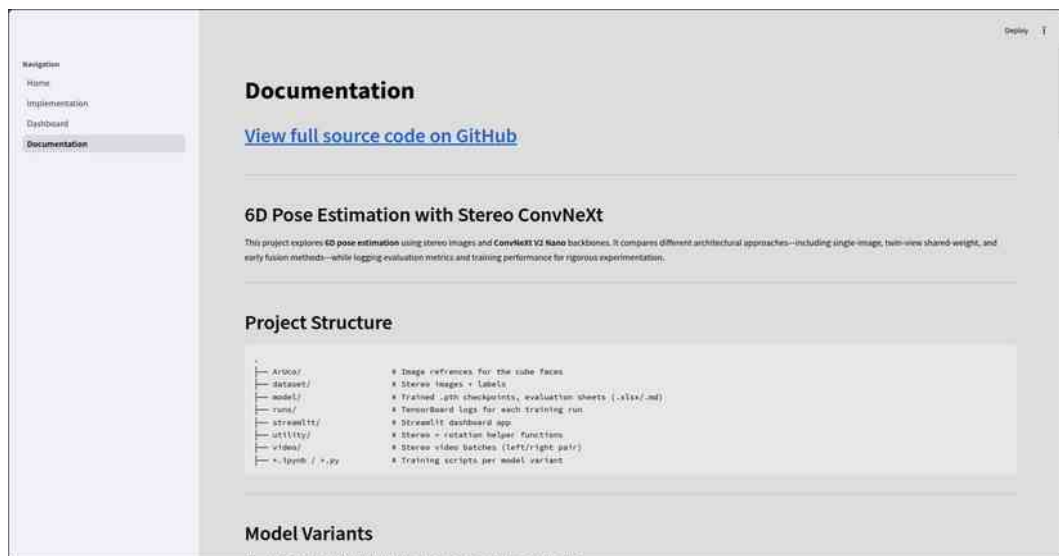
•

Double: A shared-weight twin-branch stereo model. It splits the stereo image into left and right halves, runs each through the same ComNet2 backbone (with shared weights), extracts features from both, and then concatenates the two feature vectors before feeding them into a pose regression head. This lets the model learn from stereo disparity without doubling the number of parameters.

Gambar 4.6: Implementasi Dashboard, Model Evaluation

Panel ini menampilkan tabel ringkasan metrik performa dan konfigurasi untuk seluruh varian model dan sesi pelatihan, yang bersumber dari file EVAL.xlsx. Data yang ditampilkan mencakup konfigurasi model (seperti batch size, arsitektur head, learning rate) beserta metrik evaluasi utama meliputi error translasi/rotasi, akurasi, penggunaan VRAM, dan waktu inferensi. Fasilitas download juga tersedia agar pengguna dapat mengkaji data lebih lanjut secara lokal. Di bagian bawah juga terdapat keterangan singkat mengenai masing-masing varian model yang diujikan.

Tampilan berikut merupakan halaman Documentation dari Streamlit.



Gambar 4.7: Implementasi Documentation

Pada halaman Documentation, pengguna mendapatkan akses ke penjelasan teknis proyek, struktur direktori, deskripsi tiap varian model, hingga ketergantungan perangkat lunak serta prosedur eksperimen. Informasi terstruktur ini memfasilitasi pemahaman menyeluruh terhadap desain sistem serta memudahkan replikasi maupun pengembangan lebih lanjut oleh pihak lain.

4.2 Implementasi Metode

Implementasi sistem dilakukan berdasarkan perancangan arsitektur dan alur sistem yang telah dijelaskan pada Bab 3. Model yang digunakan merupakan hasil modifikasi dari ConvNeXtV2 NANO, dan dievaluasi untuk berbagai pendekatan input gambar stereo, yaitu model Normal (gambar stereo tunggal), model EARLY FUSION (penggabungan channel), serta model Siamese Neural Network (pemrosesan gambar kiri dan kanan secara paralel dengan bobot yang sama).

4.2.1 Pemrosesan Dataset Dan Arsitektur Folder

Struktur folder penelitian ini dirancang untuk menunjang pipeline pelatihan, evaluasi, dan deployment model estimasi pose 6D berbasis gambar stereo secara modular dan terorganisasi. Berikut adalah struktur direktori utama penelitian ini:

```
SCRIPTS/
├── ArUco/ # Image references for the cube faces
├── dataset/ # Stereo images + labels batches
├── model/ # Trained .pth checkpoints, .pt, evals, etc
├── runs/ # TensorBoard logs for each training run
├── streamlit/ # Streamlit dashboard app
├── utility/ # Stereo + rotation helper functions
├── video/ # Stereo video raws (left/right pair)
├── *.ipynb / *.py # Training scripts Sanity checks etc
```

Direktori dataset/ digunakan untuk menyimpan seluruh citra latih dan uji, yang telah diproses dalam bentuk pasangan stereo terkalibrasi dan terkompresi. Selain gambar, terdapat pula metadata penting seperti informasi pembagian data (split), transformasi, serta orientasi objek. Proses ini didukung oleh skrip `aruco_dataset_builder_splitter.py` yang mengelola variabel global (misal komposisi batch, rasio pembagian data, dan augmentasi) serta penyiapan struktur folder secara otomatis, sebagai berikut:

```
# aruco_dataset_builder_splitter.py

with open("GlobVar.json", "r") as file: gv = json.load(file)
batch = gv['batch'] BATCH_ID = batch
FRAME_STEP = 1 # Skip every N frames to reduce redundancy
VAL_RATIO = 0.1
TEST_RATIO = 0.1
TRAIN_RATIO = 1 - VAL_RATIO - TEST_RATIO AUG_PER_FRAME = 1
VIDEO_PATH = "video/raw/ArUco.mp4" BATCH_FOLDER =
f"dataset/batch{BATCH_ID}"
UNSPLIT_FOLDER = os.path.join(BATCH_FOLDER, "unsplit")
IMAGES_FOLDER = os.path.join(UNSPLIT_FOLDER, "images") LABELS_PATH
= os.path.join(UNSPLIT_FOLDER, "labels.csv")
os.makedirs(IMAGES_FOLDER, exist_ok=True)
```

Dengan pendekatan ini, seluruh hasil preprocessing data menjadi lebih terstruktur dan mudah digunakan pada tahap selanjutnya. Direktori model/ bertanggung jawab sebagai repositori utama untuk seluruh artefak model. Folder ini menyimpan tiga kategori berkas utama:

- A. **Checkpoint hasil pelatihan** (misal: S-ConvNeXt6DP*.pth) yang mencakup state dict model, optimizer, dan scaler. Checkpoint disimpan otomatis ketika validasi mencapai performa terbaik selama train loop, sebagaimana dikondisikan pada kode:

```
if epoch != 0 and avg_val_loss < best_val_loss: torch.save({
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scaler_state_dict': scaler.state_dict(),
    ...
}, BEST_MODEL_PATH)
```

- B. Model telah dibersihkan dari state optimizer (CLEAN-*), yang mengandung hanya bobot model sehingga lebih efisien untuk distribusi dan inferensi.

```
CLEAN_MODEL_PATH = os.path.join(BASE_DIR, f"model/CLEAN-S-ConvNeXt6DP{BATCH_ID}.{MOD_ID}.pth")
for name, module in model.named_modules():
    if isinstance(module, torch.nn.Conv2d) or
        isinstance(module, torch.nn.Linear):
        if hasattr(module, "weight_orig"):
            prune.remove(module, "weight")
torch.save(model.state_dict(), CLEAN_MODEL_PATH)
```

- C. **Model dalam format TorchScript** (model/baked/*.pt) yang dihasilkan melalui proses tracing pada skrip Baker.py berikut:

```
scripted = torch.jit.trace(model, dummy_input)
scripted.save(outpath)
```

Model-format ini memungkinkan deployment inferensi secara cepat dan efisien pada lingkungan produksi. Selain model, folder model/ juga menyimpan seluruh hasil evaluasi performa dalam berbagai format, antara lain:

- A. File .md untuk laporan evaluasi per batch model
- B. File .csv untuk agregat hasil eksperimen
- C. File .xlsx untuk pencatatan hasil secara terstruktur

Hal ini diotomatisasi dengan kode script yang menyimpan hasil ke berbagai format berikut:

```
# .md
with open(eval_path, 'w') as f:
    f.write(eval_content)
# .csv
with open(csv_path, 'a', newline='') as csvfile:
    writer = csv.DictWriter(csvfile,
        fieldnames=csv_data.keys())
    writer.writerow(csv_data)
# .xlsx
if os.path.exists(xlsx_path):
    wb = load_workbook(xlsx_path)
else:
    wb = Workbook()
ws.append(row)
wb.save(xlsx_path)
```

Direktori runs/ menyimpan log TensorBoard untuk setiap sesi pelatihan, terorganisasi per batch (misal: S-ConvNeXt6DP_batch7.5/). Log ini mendokumentasikan perkembangan nilai loss, RMSE translasi, serta RMSE rotasi pada setiap epoch, baik untuk data latih maupun validasi:

```

writer.add_scalar("Loss/Train", avg_train_loss, epoch)
writer.add_scalar("RMSE/Train_Translation", avg_trans_rmse,
epoch)
...
writer.add_scalar("Loss/Val", avg_val_loss, epoch)
writer.add_scalar("RMSE/Val_Rotation", avg_val_rot_rmse,
epoch)

```

Pengelolaan log yang sistematis ini membantu dalam monitoring dan diagnosis kinerja model selama proses training.

Untuk keperluan uji coba dan demo berbasis web, aplikasi Streamlit dikembangkan dan ditaruh dalam direktori streamlit/, yang berisi modul-modul utama seperti Home.py, Dashboard.py, Implementation.py, dan Documentation.py.

Folder utility/ berperan sebagai penyimpanan fungsi pendukung, terutama yang berkaitan dengan konversi rotasi dan loader model berbasis TorchScript, serta berbagai utilitas lain yang dibutuhkan baik saat preprocess maupun inferensi. File notebook dan folder ArUco/ dimanfaatkan pada proses pembuatan dataset berbasis marker ArUco, yang mendukung keperluan eksperimen tambahan terhadap representasi pose objek.

Dengan demikian, struktur dan organisasi direktori ini memfasilitasi pemisahan tugas secara jelas antara pelatihan, evaluasi kuantitatif, dan demonstrasi interaktif. Selain itu, struktur modular tersebut memastikan kemudahan dalam pemeliharaan, perluasan, serta replikasi penelitian dan pengembangan sistem di masa depan. Seluruh kode, file, dan hasil eksperimen terdokumentasi secara sistematis, sehingga menunjang reproducibility serta validasi ilmiah pada penelitian ini.

4.2.1.1 Properti Dataset

Dataset yang digunakan dalam penelitian ini dibuat secara khusus untuk memenuhi kebutuhan analisis komparatif arsitektur fusi stereo. Pengambilan data dilakukan dalam lingkungan yang terkontrol untuk memastikan konsistensi dan kualitas anotasi. Proses perekaman menggunakan satu objek tunggal, yaitu sebuah kubus dengan marker ArUco di setiap sisinya, dengan latar belakang berwarna hijau polos (flat green). Kondisi pencahayaan dijaga konstan menggunakan sumber cahaya artifisial dari lampu 15 watt standar untuk meminimalkan variasi bayangan dan refleksi yang tidak diinginkan. Selama proses akuisisi, objek digerakkan secara manual di depan kamera stereo untuk menangkap berbagai macam pose dan jarak, sehingga menghasilkan dataset yang kaya akan variasi posisi dan orientasi.

Proses pembuatan dataset diotomatisasi menggunakan skrip `aruco_dataset_builder_splitter.py` untuk memastikan konsistensi. Dari video mentah, tidak semua frame diambil; skrip menggunakan parameter `FRAME_STEP` untuk mengambil gambar setiap `n` frame untuk mengurangi redundansi data antar gambar yang berurutan. Setelah semua frame yang relevan diekstraksi dan dilabeli, skrip yang sama secara programatik membagi keseluruhan data menjadi set data latih (training), validasi (validation), dan pengujian (testing) secara acak sesuai rasio yang telah ditentukan. Detail dari setiap batch dataset yang dihasilkan dan digunakan dalam penelitian ini dirangkum dalam tabel di bawah.

Tabel 4.1: Properti Dataset

Batch ID	Total Frame	Frame Latih	Frame Validasi	Frame Uji	Rasio (<i>Train: Val : Test</i>)
1	2283	1598	228	457	70:10:20
2	6799	4759	679	1361	70:10:20
3	6799	4079	1359	1361	60:20:20
4	6799	5439	679	681	80:10:10
5	21711	17368	2171	2172	80:10:10
6	14474	11579	1447	1448	80:10:10
7	7237	5789	723	725	80:10:10

4.2.2 Hyperparameter Tuning

Untuk menentukan konfigurasi optimal bagi setiap arsitektur fusi stereo yang diuji, serangkaian eksperimen dilakukan dengan memvariasikan beberapa hyperparameter kunci. Proses penyetelan ini bertujuan untuk menemukan keseimbangan terbaik antara akurasi model dan efisiensi penggunaan sumber daya komputasi. Analisis ini didasarkan pada data hasil eksperimen yang terdokumentasi dalam EVAL.xlsx, yang merekam dampak dari setiap perubahan konfigurasi terhadap metrik performa.

- A. Ukuran Citra Input (*Image Size*) Eksperimen dilakukan pada dua resolusi *input* utama: 224x224 dan 384x384 piksel. Hasil analisis menunjukkan bahwa penggunaan resolusi 384x384 secara umum mampu menghasilkan error translasi (RMSE) yang lebih rendah. Namun, peningkatan akurasi ini

datang dengan biaya komputasi yang signifikan, di mana penggunaan VRAM meningkat rata-rata dari 5.1 GB menjadi 8.2 GB dan waktu pelatihan per sesi hampir dua kali lipat. Mengingat keterbatasan perangkat keras (GPU 8 GB) dan fokus penelitian pada solusi yang efisien, resolusi 224x224 dipilih sebagai standar untuk sebagian besar eksperimen komparatif karena menawarkan keseimbangan yang lebih baik antara performa dan kebutuhan sumber daya.

- B. *Arsitektur Head* Beberapa variasi arsitektur head diuji coba, mulai dari yang sederhana (misalnya, satu *hidden layers* $640 \rightarrow 512 \rightarrow 9$) hingga yang lebih kompleks (misalnya, dua hidden layers $640 \rightarrow 256 \rightarrow 128 \rightarrow 9$). Data menunjukkan bahwa arsitektur *head* yang lebih sederhana cenderung lebih stabil dan memberikan hasil yang lebih baik secara konsisten. Sebagai contoh, pada varian baseline, head dengan arsitektur $640 \rightarrow 512 \rightarrow 9$ mencapai VAL_LOSS rata-rata 0.42, sementara arsitektur yang lebih dalam ($640 \rightarrow 256 \rightarrow 128 \rightarrow 9$) menghasilkan VAL_LOSS yang jauh lebih tinggi yaitu 1.91, mengindikasikan adanya overfitting atau ketidakstabilan pelatihan. Temuan ini mengarahkan pemilihan arsitektur head yang lebih dangkal untuk sebagian besar model final.
- C. *Ukuran Batch dan Laju Pembelajaran* Ukuran batch divariasikan antara 32, 48, hingga 96. Pemilihan ukuran batch akhir untuk setiap model merupakan kompromi antara stabilitas gradien dan batas VRAM yang tersedia. Laju pembelajaran (learning rate) ditetapkan konstan pada 0.0001 setelah beberapa pengujian awal menunjukkan bahwa nilai ini

memberikan konvergensi yang stabil di sebagian besar eksperimen tanpa menyebabkan lonjakan *loss*.

- D. *Epoch* dan *Patience* Jumlah *epoch* tidak ditetapkan secara kaku, melainkan dikontrol melalui mekanisme *early stopping* dengan nilai *patience* sebesar 3. Ini berarti pelatihan akan berhenti jika tidak ada perbaikan pada *validation loss* selama tiga *epoch* berturut-turut. Analisis data menunjukkan bahwa jumlah *epoch* bervariasi antara 20 hingga 30 di seluruh eksperimen, yang membuktikan bahwa setiap model dilatih hingga mencapai titik konvergensi optimalnya masing-masing tanpa membuang sumber daya komputasi atau mengalami *overfitting* yang berlebihan.

Berdasarkan serangkaian proses penyetelan ini, konfigurasi hyperparameter yang paling optimal untuk setiap varian arsitektur berhasil diidentifikasi, yang kemudian digunakan sebagai dasar untuk perbandingan akhir di bagian hasil penelitian.

4.2.3 Arsitektur Model

Arsitektur model yang digunakan dalam penelitian ini berfokus pada eksplorasi pemanfaatan informasi stereo dalam tugas estimasi pose 6D berbasis deep learning. Secara khusus, diterapkan tiga varian arsitektur yang seluruhnya berbasis backbone ConvNeXt V2 NANO, dimana setiap arsitektur dirancang untuk merepresentasikan pendekatan pemrosesan data dan ekstraksi fitur stereo yang berbeda. Pemilihan ConvNeXt V2 NANO sebagai backbone didasarkan pada pertimbangannya sebagai model vision transformer modern yang relatif

ringan, namun memiliki kapabilitas representasi fitur spasial yang baik. Adapun ketiga varian arsitektur yang dievaluasi adalah sebagai berikut:

A. *Baseline*

Pada varian ini, model menerima satu citra RGB sebagai input. Struktur dasarnya memanfaatkan seluruh arsitektur ConvNeXtV2-NANO hingga sebelum lapisan klasifikasi akhir, kemudian ditambahkan kepala regresi (regression head) beroutput sembilan dimensi, yang terdiri atas tiga parameter translasi (t_x , t_y , t_z) serta enam parameter rotasi menggunakan representasi ortho6D. Perancangan kepala regresi dapat diilustrasikan melalui potongan kode berikut:

```
self.head = nn.Sequential(
    nn.Linear(640, 512),
    nn.ReLU(),
    nn.Linear(512, 9)
)
```

Pendekatan ini digunakan sebagai baseline konvensional, di mana gambar RGB tunggal menjadi satu-satunya sumber informasi spasial maupun tekstural bagi model. Model semacam ini memang sederhana dan rendah kompleksitas komputasi, karena tidak perlu melakukan proses agregasi informasi antara dua sumber (seperti stereo). Namun, kekurangannya terletak pada kurangnya informasi kedalaman dengan resolusi tinggi atau eksplisit yang disediakan oleh sistem penglihatan stereo. Dengan demikian, varian ini sangat penting untuk menguji efektivitas penambahan modalitas stereo (pada eksperimen berikutnya) dibandingkan penggunaan citra monokular.

B. Jaringan Saraf Siamese (*Siamese Neural Network*)

Pada varian ketiga, digunakan pendekatan dua jalur terpisah (twin stream) dengan konsep berbagi bobot (shared weights). Masing-masing gambar stereo (kiri dan kanan) diproses secara independen menggunakan backbone ConvNeXtV2-NANO yang berbagi parameter, dan fitur representatif yang dihasilkan dari kedua stream digabungkan sebelum masuk ke kepala regresi akhir. Penggabungan fitur dilakukan melalui operasi konkatenasi sebagai berikut:

```
f = torch.cat([feat_left, feat_right], dim=1)
self.head = nn.Sequential(
    nn.Linear(640 * 2, 256),
    nn.ReLU(),
    nn.Linear(256, 9)
)
```

Desain ini dirancang untuk memaksimalkan kekuatan komparatif antara kedua input stereo-backbone yang sama memungkinkan efisiensi parameter model, sementara proses pemisahan memungkinkan jaringan untuk benar-benar “membandingkan” fitur dari sisi kiri dan kanan sebelum keputusan regresi pose dilakukan. Representasi yang dihasilkan menjadi lebih kaya, karena masing-masing sisi mempertahankan distingsinya sampai tahap ekstraksi fitur yang cukup dalam. Namun, pendekatan ini membawa konsekuensi pada peningkatan beban komputasi (karena fitur dari dua backbone dikalkulasi) serta kecepatan inferensi yang relatif lebih lambat dibanding EARLY FUSION.

C. Fusi Dini (*Early Fusion*)

Varian kedua bertujuan mengeksplorasi metode “early fusion” informasi stereo, di mana gambar kiri dan kanan dari pasangan stereo digabungkan langsung pada tingkat channel input. Masing-masing channel warna

(RGB) dari gambar kiri dan kanan dikombinasikan, membentuk tensor input dengan 6 channel. Untuk mengakomodasi perubahan ini, lapisan konvolusi pertama (first conv layer) pada ConvNeXtV2-NANO dimodifikasi agar menerima 6 channel alih-alih 3 channel:

```
new_conv = nn.Conv2d(in_channels=6,
                      out_channels=old_conv.out_channels,
                      ...)
```

Dengan desain ini, backbone ConvNeXt selanjutnya akan memproses citra stereo sebagai satu entitas, sehingga model diharapkan mampu mengekstraksi kedalaman secara implisit melalui informasi disparitas antar-channel kiri dan kanan. Keunggulan utama dari pendekatan ini adalah efisiensi; karena seluruh pemrosesan melalui satu stream backbone. Namun, potensi kendalanya adalah risiko “campur aduk” (interferensi) informasi stereo, sebab jaringan harus belajar memilah mana informasi yang berasal dari gambar kiri dan kanan sejak tahap awal, tanpa mekanisme eksplisit yang membedakan kontribusi masing-masing input.

Tabel berikut merangkum perbedaan inti ketiga varian model:

Tabel 4.2: Arsitektur Regression Head

Model	Jenis Input	Informasi Kedalaman	Kompleksitas	Kecepatan
Baseline	Satu citra stereo	Sedikit	Ringan	Cepat

	RGB			
EARLY FUSION	Multi Channel Image (6 channel)	Tersirat (implisit)	Sedang	Sedang
SNN	Stereo (2 input)	Eksplisit, jelas	Lebih berat	Lebih lambat

4.2.4 Konversi Ke TorchScript

Setelah proses pelatihan selesai dan model terbaik diperoleh dalam format .pth, langkah selanjutnya adalah melakukan konversi model ke dalam format yang lebih sesuai untuk deployment, yaitu .pt dengan menggunakan TorchScript. Proses ini dikenal dengan istilah "model freezing" dan bertujuan untuk menyatukan arsitektur model dan bobot terlatih dalam satu berkas eksekusi mandiri. Perbedaan mendasar antara format .pth dan .pt terletak pada isi dan ketergantungannya: format .pth hanya memuat bobot model dan tetap memerlukan inisialisasi kelas model pada saat inferensi, sedangkan format .pt (TorchScript) telah mengkompilasi baik arsitektur maupun bobot sehingga dapat langsung digunakan, meningkatkan efisiensi loading dan kompatibilitas lintas platform.

Konversi dilakukan dengan fungsi `torch.jit.trace()` pada model yang telah dibersihkan dari status optimizer, menggunakan dummy input yang sesuai dengan arsitektur. Hasilnya adalah model statis yang dapat dieksekusi pada berbagai

lingkungan runtime tanpa kebutuhan terhadap kode sumber model. Model hasil freezing ini memiliki sejumlah keunggulan utama:

- A. *Loading Time* berkurang secara drastis
- B. Tidak ketergantungan pada definisi kelas model dan versi *library* tertentu
- C. Kompatibilitas luas, termasuk untuk aplikasi Python maupun C++
- D. Efisiensi inferensi karena model telah terkompilasi dalam bentuk statis.

4.2.5 Sistem Evaluasi Dan Metrik Evaluasi

Sistem evaluasi dalam penelitian ini dirancang untuk memberikan analisis kuantitatif yang komprehensif terhadap kinerja setiap arsitektur model. Evaluasi ini berfokus pada tiga pilar utama: akurasi prediksi, efisiensi inferensi, dan penggunaan sumber daya. Setiap metrik dipilih untuk memberikan gambaran yang jelas tentang kelayakan model untuk implementasi di dunia nyata, dengan penjabaran detail bagaimana nilai output model diolah menjadi hasil evaluasi yang terukur.

Penjabaran akurasi prediksi adalah inti dari evaluasi ini. Model menghasilkan output berupa vektor 9 variabel, yang kemudian dibandingkan dengan data ground truth untuk menghitung Root Mean Square Error (RMSE). Untuk mengilustrasikan proses ini, mari kita ambil satu sampel data aktual dari set validasi (labels.csv):

Data Aktual (Ground Truth): [0.0488, -0.0155, 0.3698, 0.2171, -0.1774, -0.8032, -0.5912, -0.5546, 0.7867]

Misalkan setelah memproses gambar `frame_4109_v0.png`, model memberikan output estimasi sebagai berikut:

Data Estimasi (Prediksi Model): [0.0512, -0.0140, 0.3711, 0.2201, -0.1750, -0.8010, -0.5900, -0.5530, 0.7890]

Akurasi translasi dihitung dari tiga variabel pertama (x, y, z). Kedua set nilai ini dimasukkan ke dalam fungsi `calculate_translation_rmse` untuk menghitung selisih jarak geometrisnya. Fungsi ini menghitung akar dari rata-rata jumlah kuadrat selisih antara koordinat prediksi dan aktual, lalu mengonversinya ke sentimeter. Perhitungan ini memberikan satu nilai RMSE tunggal yang merepresentasikan rata-rata kesalahan posisi dalam satuan yang mudah dipahami.

```
def calculate_translation_rmse(preds, gts):
    pred_trans = preds[:, :3]
    gt_trans = gts[:, :3]
    trans_rmse = np.sqrt(np.mean(np.sum((pred_trans -
gt_trans)**2, axis=1)))
    return trans_rmse * 100
```

Sementara itu, akurasi rotasi dihitung dari enam variabel berikutnya.

Karena variabel ini berada dalam format representasi rotasi kontinu 6D, keduanya harus terlebih dahulu dikonversi menjadi matriks rotasi 3x3 yang valid secara matematis. Proses ini menjamin bahwa orientasi yang dihasilkan ortonormal dan bebas dari ambiguitas. Setelah kedua representasi 6D (aktual dan estimasi) diubah menjadi matriks rotasi, selisih sudut antara keduanya dihitung. RMSE rotasi adalah nilai rata-rata dari selisih sudut ini, yang dinyatakan dalam derajat.

```
# Mengkonversi representasi 6D ke matriks rotasi 3x3
def compute_rotation_matrix_from_ortho6d(poses_6d):
    x_raw = poses_6d[:, 0:3]
    y_raw = poses_6d[:, 3:6]
    x = F.normalize(x_raw, dim=1)
    z = F.normalize(torch.cross(x, y_raw, dim=1), dim=1)
    y = torch.cross(z, x, dim=1)
    return torch.stack((x, y, z), dim=-1)
# Menghitung selisih sudut rotasi dalam derajat
def rotation_error(R_pred, R_gt):
```

```

R_diff = torch.bmm(R_pred.transpose(1, 2), R_gt)
trace = torch.diagonal(R_diff, dim1=1, dim2=2).sum(dim=1)
angle_rad = torch.acos(torch.clamp((trace - 1) / 2, min=-1.0,
max=1.0))
return torch.rad2deg(angle_rad)

```

Selain akurasi, efisiensi juga merupakan aspek penting. Waktu inferensi diukur untuk mengetahui kecepatan model. Pengukuran ini dilakukan dengan mencatat waktu sesaat sebelum dan sesudah model dijalankan pada satu batch data, lalu hasilnya dibagi dengan jumlah data dalam batch tersebut untuk mendapatkan waktu per gambar dalam milidetik. Kode berikut dari fungsi validasi menunjukkan implementasinya, di mana `torch.cuda.synchronize()` digunakan untuk memastikan pengukuran yang akurat pada GPU.

```

# Menghitung waktu inferensi per gambar dalam milidetik
torch.cuda.synchronize()
start_time = time.time()
# Jalankan prediksi model
outputs = model(images)
torch.cuda.synchronize()
inference_time = (time.time() - start_time) * 1000 /
images.size(0)

```

Terakhir, penggunaan sumber daya VRAM diukur untuk menilai efisiensi memori model. Pengukuran ini dilakukan dengan memantau proses eksekusi Python pada utilitas NVIDIA System Management Interface (`nvidia-smi`) dan mencatat nilai maksimum pada kolom "GPU Memory Usage". Ini memberikan gambaran jelas mengenai kebutuhan perangkat keras untuk menjalankan model, yang sangat relevan untuk penempatan pada perangkat dengan sumber daya terbatas.

4.2.6 Hyperparameter Tuning

Pemilihan *hyperparameter* yang optimal merupakan langkah krusial yang secara langsung memengaruhi performa dan efisiensi model deep learning. Daripada menggunakan satu set parameter yang kaku, penelitian ini mengadopsi pendekatan penyetelan empiris dan iteratif. Serangkaian eksperimen dilakukan dengan memvariasikan hyperparameter kunci secara terkontrol untuk mengamati dampaknya terhadap metrik evaluasi utama: *Root Mean Square Error* (RMSE) untuk akurasi, waktu inferensi untuk kecepatan, dan penggunaan VRAM untuk efisiensi sumber daya. Proses ini bertujuan untuk menemukan bukan hanya performa puncak dari setiap arsitektur, melainkan sebuah konfigurasi yang seimbang dan adil sebagai dasar perbandingan akhir. Seluruh hasil dari proses ini direkam secara sistematis dalam EVAL.xlsx.

Hyperparameter	Nilai yang Diuji	Catatan
Ukuran Citra	224x224, 384x384	Menguji keseimbangan antara detail visual dan beban komputasi.
Ukuran Batch	32, 48, 64, 96	Disesuaikan berdasarkan arsitektur dan batasan VRAM.
Arsitektur Head	640 → 512 → 9 640 → 256 → 9 640x2 → 256 → 9 640 → 256 → 128 → 9	Eksplorasi kedalaman head untuk model single-stream dan dual-stream.
Dataset	Batch 5, 6, 7	Menggunakan dataset dengan ukuran dan komposisi yang berbeda.

Setelah melalui serangkaian eksperimen eksploratif ini, satu set hyperparameter final dipilih untuk melakukan perbandingan yang adil dan terkontrol antara ketiga arsitektur. Tujuan dari standardisasi ini adalah untuk memastikan bahwa perbedaan performa yang terukur benar-benar disebabkan oleh perbedaan strategi fusi arsitektur, bukan karena keuntungan dari konfigurasi hyperparameter yang berbeda. Berikut adalah justifikasi untuk setiap parameter final yang dipilih:

- A. **Dataset** (Batch 7): Batch 7 dipilih sebagai dataset standar karena ukurannya yang representatif dan kualitasnya yang bersih tanpa augmentasi, menyediakan dasar yang konsisten untuk menguji ketiga arsitektur.
- B. **Ukuran Citra** (224x224): Meskipun resolusi 384x384 menunjukkan akurasi yang sedikit lebih tinggi, resolusi 224x224 dipilih karena menawarkan keseimbangan terbaik antara akurasi dan efisiensi sumber daya, memastikan semua model dapat dilatih dan dievaluasi dalam batasan VRAM 8GB.
- C. **Laju Pembelajaran** (0.0001): Nilai ini secara konsisten menunjukkan konvergensi yang stabil di sebagian besar eksperimen tanpa menyebabkan lonjakan loss atau pelatihan yang terlalu lambat.
- D. **Arsitektur Head** (...→256→9): Analisis data pada EVAL.xlsx menunjukkan bahwa head dengan satu lapisan tersembunyi (single hidden layer) dengan 256 neuron (640→256→9 untuk baseline & early fusion,

640x2→256→9 untuk SNN) lebih stabil dan kurang rentan terhadap overfitting dibandingkan dengan arsitektur head yang lebih dalam.

- E. **Ukuran Batch** (48): Ukuran batch 48 dipilih sebagai nilai standar. Ini adalah ukuran batch terbesar yang dapat ditangani oleh arsitektur SNN (yang paling intensif memori) dalam batasan perangkat keras yang ada, sehingga memungkinkan perbandingan yang setara di antara ketiga model. Di atas ukuran batch 48, pytorch mengalami Error OOM.
- F. **Kontrol Pelatihan** (Patience=3): Penggunaan early stopping dengan patience 3 memastikan bahwa setiap model dilatih hingga mencapai titik konvergensi optimalnya, yang merupakan metode perbandingan yang lebih adil daripada menggunakan jumlah epoch yang tetap.

4.2.7 Implementasi Streamlit

Implementasi antarmuka Streamlit dalam penelitian ini berperan sebagai portal interaktif untuk menguji, memvisualisasikan, dan membandingkan model-model estimasi pose 6D berbasis stereo vision. Streamlit dipilih karena kemampuannya menghadirkan workflow machine learning secara “end-to-end” mulai dari pengujian langsung, visualisasi 3D, pemantauan performa tiap eksperimen, hingga dokumentasi terintegrasi dalam platform web yang mudah diakses dan diperluas. Berikut adalah analisis mendalam terkait fungsionalitas dan arsitektur aplikasi Streamlit pada submodul utama:

Pada `Implementation.py`, UI Streamlit menyediakan beberapa mode input yang bervariasi:

- A. Webcam melalui WebRTC (Stereo Webcam): Menggunakan modul `streamlit_webrtc`, aplikasi dapat menangkap frame stereo real-time dari webcam pengguna. Modul ini memanfaatkan objek `StereoVideoProcessor(VideoProcessorBase)`, yang melakukan pengambilan frame, inferensi pose menggunakan model `TorchScript`, dan pengiriman ulang frame dengan overlay prediksi ke UI.
- B. Upload Video: Pengguna dapat mengunggah citra atau video stereo (format side-by-side) untuk dievaluasi. Proses ini memanfaatkan fungsi `st.file_uploader()` dan pipeline inferensi identik dengan pipeline real-time.
- C. Local Video: Akses ke video stereo di folder `video/raw` melalui dialog selection.
- D. Upload Image: Pengujian cepat berbasis satu citra stereo.

Penggabungan berbagai mode input membantu memastikan sistem dapat diuji dalam berbagai senario nyata (real-time maupun batch).

Potongan kode utama:

```
mode = st.radio("Input Source", ["Stereo Webcam",
                                "Upload Video",
                                "Use Local Video",
                                "Upload Image"])
```

dan untuk processing kamera:

```
ctx = webrtc_streamer(
    key="stereo-webrtc",
    video_processor_factory=StereoVideoProcessor,
    media_stream_constraints={"video": {"width": 1280,
                                        "height": 480}},
    async_processing=True,
```

```
)
```

Pemilihan Model dan Loader TorchScript

Aplikasi menyediakan pemilih model yang secara dinamis mendeteksi berkas .pt di folder model hasil proses freezing (TorchScript), dan memuat model terpilih secara instan di device (CPU/GPU).

```
model_file = st.selectbox("Select Model",
                           get_available_models())
model = inference_model.get_model(os.path.join(MODEL_DIR,
                                                model_file))
```

Dengan ini, pengguna dapat membandingkan performa beberapa arsitektur (Baseline, SNN, EARLY FUSION) tanpa restart aplikasi.

Inferensi dan Visualisasi Real-time

Pipeline inferensi dan visualisasi terdiri dari beberapa langkah otomatis:

- A. Preprocessing: Citra stereo diparsing menjadi input yang sesuai dengan arsitektur, misalnya, kanan/kiri diproses terpisah untuk model shared-weight, atau digabung menjadi 6 channel untuk early fusion.
- B. Inferensi Model: Model menjalankan prediksi pose 6D pada batch input (bisa satu gambar atau pasangan stereo).
- C. Visualisasi 3D: Fungsi `render_3d_pose()` memanfaatkan Open3D untuk membangun objek kubus berwarna yang dipose sesuai hasil prediksi. Kubus virtual ini digambarkan pada canvas dan diubah posisinya menggunakan transformasi prediksi model (translasi dan rotasi).

Contoh kode pipeline:

```
def predict_and_render(model, frame):
    ...
    output = model(stacked) # atau model(left_img, right_img)
    render = render_3d_pose(output)
    return combined, render
```

dan untuk Visualisasi 3D:

```
def render_3d_pose(pose):
    # translate, rotate, gunakan Open3D untuk membuat mesh
    kubus
    ...
    cube.rotate(R, center=(0, 0, 0))
    cube.translate(trans)
    ...
    return (np.asarray(img) * 255).astype(np.uint8)
```

Monitoring & Evaluasi Eksperimen

Dashboard.py merangkum seluruh workflow pemantauan eksperimen, antara lain:

- A. Perbandingan Training Runs: Visualisasi metrik (Loss/Train, RMSE/Train_Translation dsb.) lintas berbagai training runs melalui integrasi TensorBoard event accumulator dan plot matplotlib, memungkinkan pemantauan progres dan analisis tren performa tiap batch eksperimen.

```
metric = st.selectbox("Select Metric", [...])
selected_runs = st.multiselect("Select Runs to Compare", runs, default=runs)
fig = plot_selected_metric(metric, selected_runs)
```

```
st.pyplot(fig)
```

- B. Ringkasan Evaluasi: Data agregat hasil evaluasi setiap model varian dapat ditampilkan dan diunduh sebagai .csv (EVAL.xlsx).
- C. Laporan Model Markdown: Otomatisasi pemuatan hasil eksperimen per-model (.md), memudahkan studi post-mortem serta pelaporan.

Integrasi Dokumentasi dan Dataset

Documentation.py serta panel Dataset Parameters Sheet pada Dashboard memberikan informasi ringkas namun menyeluruh terkait:

- A. Struktur dan isi dataset, meliputi info batch, metadata per batch, distribusi label, dan seterusnya.
- B. Ketergantungan perangkat lunak, arsitektur model, strategi pelatihan dan evaluasi.
- C. Panduan navigasi dan kode sumber dalam repositori.

Implementasi Streamlit pada penelitian ini mengintegrasikan seluruh workflow dari pelatihan, pengujian, analisis dataset, hingga visualisasi pose dalam satu aplikasi web yang modular dan dapat diperluas. Kemampuan UI untuk mendemonstrasikan inferensi real-time, menampilkan visualisasi kuantitatif (grafik, statistik), dan memungkinkan pemilihan bebas antar model (melalui format TorchScript) menegaskan peran dashboard sebagai alat eksperimental dan diagnostik.

Keunggulan konseptual desain ini memungkinkan:

- A. Reprodusibilitas eksperimen oleh pihak lain (transparansi interaksi input-output dan eksplisitnya parameter yang digunakan).
- B. Analisis multi-varian yang objektif, didukung oleh visualisasi data hasil yang mudah dieksplorasi.
- C. Modularitas, setiap panel baik untuk monitoring loss, inspeksi dataset, maupun pengujian model, dapat dikembangkan terpisah.

Dengan demikian, UI Streamlit pada penelitian estimasi pose 6D ini bukan hanya sebagai alat demonstrasi, namun juga sebagai jembatan kritis untuk integrasi workflow penelitian, *monitoring* performa, dokumentasi, serta pengujian interaktif untuk mendukung pengambilan keputusan desain arsitektur model secara lebih informatif dan terfokus.

4.3 Hasil Akhir Dan Analisis Komparatif

Setelah melalui proses penyetelan hyperparameter untuk menemukan konfigurasi yang paling adil, ketiga arsitektur fusi stereo—Baseline (BASE), Siamese Neural Network (SNN), dan Early Fusion (EF)—diuji menggunakan satu set parameter yang terstandarisasi. Pengujian ini bertujuan untuk mengisolasi dan membandingkan secara langsung dampak dari strategi fusi itu sendiri terhadap performa model. Seluruh model dilatih pada Dataset Batch 7 dengan ukuran citra 224x224, laju pembelajaran 0.0001, dan kontrol pelatihan early stopping dengan

patience 3. Ukuran batch ditetapkan pada 48, yang merupakan batas maksimal untuk arsitektur SNN sebelum mengalami error Out-of-Memory (OOM).

4.3.1 Analisis Performa Agregat

Analisis pertama berfokus pada performa agregat dari setiap arsitektur pada seluruh data validasi. Tabel di bawah ini menyajikan data mentah dari hasil pengujian, yang mencakup metrik akurasi (RMSE), penggunaan VRAM, dan kecepatan inferensi.

Tabel 4.3: Data Mentah Hasil Validasi Komparatif

Varian Arsitektur	RMSE Translasi (m)	RMSE Rotasi (°)	Penggunaan VRAM (GB)	Waktu Inferensi (ms)
SNN	0.0577	39.0942	7.0	2.06
EF (Early Fusion)	0.0600	40.3100	4.0	1.11
BASE (Baseline)	0.0612	41.8812	4.0	1.05

Untuk memahami signifikansi dari hasil di atas, dilakukan analisis perbedaan persentase di mana performa terburuk pada setiap metrik dijadikan

sebagai titik acuan (peningkatan 0%). Analisis ini memperjelas keunggulan relatif dari setiap arsitektur.

Tabel 4.4: Analisis Persentase Peningkatan Performa

Varian Arsitektur	Peningkatan Akurasi Translasi	Peningkatan Akurasi Rotasi	Peningkatan Efisiensi VRAM	Peningkatan Kecepatan Inferensi
SNN	+5.72%	+6.65%	0.00%	0.00%
EF (Early Fusion)	+1.96%	+3.75%	+42.86%	+46.12%
BASE (Baseline)	0.00%	0.00%	+42.86%	+49.03%

Dari analisis agregat, terlihat jelas bahwa arsitektur **SNN** secara konsisten menghasilkan akurasi tertinggi, namun dengan konsumsi sumber daya yang paling boros. Sebaliknya, arsitektur **Baseline** dan **Early Fusion** jauh lebih efisien dalam hal VRAM dan kecepatan.

4.3.2 Analisis Kualitatif Pada Sampel Prediksi

Untuk memberikan gambaran yang lebih konkret mengenai bagaimana perbedaan akurasi ini bermanifestasi pada level prediksi tunggal, dilakukan perbandingan output mentah 9-variabel dari ketiga model terhadap satu sampel gambar (frame_0002_v0.png). Tabel berikut secara langsung menyajikan perbandingan nilai aktual (ground truth) dengan nilai estimasi dari setiap model.

Source	z	y	x	r1x	r1y	r1z	r2x	r2y	r2z
GROUND TRUTH	- 0.0548	0.0981	0.5040	0.9427	0.1701	0.1901	- 0.9808	0.2741	0.0953
BASELINE	- 0.1818	0.1334	0.5823	1.3433	- 0.2582	- 0.4193	0.3570	- 0.9596	1.4916
SNN	- 0.0540	0.0913	0.5186	0.9442	0.1751	0.2227	- 0.8660	0.3204	0.1704
EARLY FUSION	- 0.0700	0.1011	0.5190	0.9890	0.1764	0.1651	- 0.8756	0.2985	0.1585

Dari tabel di atas, dapat dilihat secara kualitatif bagaimana nilai RMSE agregat terbentuk. Nilai translasi (**x**, **y**, **z**) dari model **SNN** terlihat paling mendekati nilai *GROUND_TRUTH*. Sebaliknya, model **BASELINE**

menunjukkan deviasi yang paling signifikan pada hampir semua variabel, baik translasi maupun rotasi. Model **EARLY_FUSION** berada di posisi tengah, dengan prediksi yang cukup mendekati *ground truth* meskipun tidak seakurat SNN, namun jauh lebih baik daripada *Baseline*. Perbandingan konkret ini secara efektif menjawab kebutuhan untuk memahami bagaimana kesalahan prediksi direpresentasikan pada level output mentah.

4.3.3 Ringkasan Analisis

Berdasarkan analisis agregat dan kualitatif, dapat disimpulkan bahwa setiap arsitektur menawarkan keunggulan yang berbeda. Arsitektur SNN adalah yang **paling akurat**, namun paling tidak efisien. Arsitektur **Baseline** adalah yang **paling efisien**, namun paling tidak akurat. Dengan demikian, arsitektur **Early Fusion** muncul sebagai solusi yang menawarkan **keseimbangan (*trade-off*) terbaik** antara akurasi dan efisiensi, menjadikannya kandidat terkuat untuk aplikasi dunia nyata yang menuntut performa *real-time* pada perangkat keras dengan sumber daya terbatas. Temuan ini sejalan dengan kebutuhan di bidang robotika, di mana pemilihan arsitektur yang tepat merupakan kunci untuk tugas-tugas manipulasi objek yang andal [22].

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan analisis komparatif dan evaluasi yang telah dilakukan terhadap tiga arsitektur fusi stereo untuk estimasi pose 6D, dapat ditarik beberapa kesimpulan yang secara langsung menjawab pertanyaan-pertanyaan penelitian:

- A. Perbandingan Performa Arsitektur:** Terdapat perbedaan performa yang signifikan antara ketiga arsitektur fusi stereo yang diuji. Arsitektur **Jaringan Siamese (SNN)** sebagai representasi *late fusion* menunjukkan akurasi tertinggi. Di sisi lain, arsitektur **Baseline** dan **Early Fusion**, yang keduanya memproses data melalui satu alur (*single stream*), menunjukkan efisiensi komputasi (kecepatan inferensi dan penggunaan VRAM) yang jauh lebih unggul.
- B. Arsitektur dengan Akurasi Tertinggi:** Dari ketiga arsitektur, **Jaringan Siamese (SNN)** secara konsisten menghasilkan tingkat akurasi tertinggi, dengan nilai RMSE translasi sebesar **0.0577 m** dan RMSE rotasi sebesar **39.09°**. Hal ini membuktikan bahwa strategi *late fusion*, yang memproses

fitur dari kedua gambar secara independen sebelum menggabungkannya, paling efektif dalam mengekstraksi informasi spasial yang presisi.

C. Keseimbangan (*Trade-off*) antara Akurasi dan Efisiensi: Setiap arsitektur menawarkan keseimbangan yang berbeda. Arsitektur SNN unggul dalam akurasi namun paling boros sumber daya (VRAM 7.0 GB, inferensi 2.06 ms). Arsitektur Baseline adalah yang paling efisien (VRAM 4.0 GB, inferensi 1.05 ms) namun paling tidak akurat. Arsitektur **Early Fusion** muncul sebagai solusi yang menawarkan **keseimbangan terbaik**, dengan akurasi yang hanya sedikit di bawah SNN (RMSE translasi 0.0600 m) namun dengan efisiensi yang hampir setara dengan Baseline (VRAM 4.0 GB, inferensi 1.11 ms).

D. Keterbatasan dan Karakteristik Unik: Penelitian ini mengidentifikasi bahwa meskipun akurat, arsitektur SNN memiliki keterbatasan praktis untuk aplikasi *real-time* karena kebutuhan VRAM yang tinggi. Sebaliknya, arsitektur *single-stream* seperti Early Fusion lebih cocok untuk implementasi pada perangkat dengan sumber daya terbatas. Namun, perlu dicatat bahwa semua model dalam penelitian ini diuji dalam kondisi pencahayaan yang terkontrol dan dengan satu objek tunggal. Kinerja model di dunia nyata dapat dipengaruhi oleh faktor-faktor seperti oklusi

(objek terhalang), perubahan pencahayaan yang drastis, dan latar belakang yang kompleks, yang menjadi batasan dari penelitian ini.

Secara keseluruhan, penelitian ini menyimpulkan bahwa **arsitektur *Early Fusion* (fusi dini)** merupakan strategi yang paling optimal untuk estimasi pose 6D dari video stereoskopik, karena mampu memberikan kombinasi terbaik antara akurasi yang tinggi dan efisiensi komputasi yang sangat baik.

5.2 Saran

Berdasarkan hasil analisis dan kesimpulan yang telah dijabarkan, terdapat beberapa saran yang dapat menjadi acuan untuk pengembangan dan penelitian di masa mendatang guna meningkatkan akurasi dan efisiensi sistem estimasi pose 6D berbasis visi stereo:

A. Eksplorasi Arsitektur *Intermediate Fusion*: Penelitian ini telah membandingkan *early fusion* dan *late fusion*. Langkah selanjutnya yang logis adalah mengeksplorasi arsitektur *intermediate fusion*, di mana fitur dari kedua gambar digabungkan pada lapisan tengah *backbone*. Pendekatan ini secara teoretis dapat menggabungkan keunggulan dari kedua metode, yaitu memanfaatkan korelasi spasial tingkat rendah seperti

early fusion sambil tetap memungkinkan ekstraksi fitur independen seperti *late fusion*.

B. Penggunaan Dataset yang Lebih Kompleks dan Bervariasi: Model

dalam penelitian ini dilatih pada dataset dengan objek tunggal dalam kondisi terkontrol. Untuk meningkatkan robustisitas dan generalisasi model, penelitian selanjutnya disarankan untuk menggunakan dataset yang lebih menantang. Dataset tersebut dapat mencakup beberapa objek sekaligus (*multi-object*), adanya oklusi (objek terhalang sebagian), variasi pencahayaan yang drastis, serta latar belakang yang dinamis.

C. Optimasi *Hyperparameter* Lanjutan: Proses penyetelan *hyperparameter*

dalam penelitian ini dilakukan secara empiris untuk mendapatkan konfigurasi yang adil. Untuk mendapatkan performa puncak dari arsitektur *Early Fusion* yang terbukti paling seimbang, dapat diterapkan teknik optimasi *hyperparameter* otomatis seperti *Grid Search*, *Random Search*, atau *Bayesian Optimization*. Teknik ini dapat menemukan kombinasi parameter yang lebih optimal untuk meningkatkan akurasi lebih lanjut.

D. Validasi pada Perangkat Keras Berbeda: Pengujian efisiensi dalam

penelitian ini dilakukan pada satu jenis GPU. Untuk validasi di dunia nyata, sangat disarankan untuk melakukan *benchmarking* model *Early Fusion* pada berbagai perangkat keras dengan sumber daya terbatas,

seperti NVIDIA Jetson, Raspberry Pi, atau prosesor pada *smartphone*. Hal ini akan memberikan data performa yang lebih konkret untuk aplikasi *edge computing*.

E. Penanganan Objek dengan Karakteristik Khusus: Penelitian ini berfokus pada objek solid dan tidak transparan. Penelitian selanjutnya dapat mengadaptasi arsitektur fusi yang paling seimbang (Early Fusion) untuk menangani kasus yang lebih menantang seperti objek transparan, yang seringkali gagal diproses oleh sensor kedalaman konvensional. Pendekatan seperti yang diusulkan oleh Chen et al. dapat menjadi titik awal yang baik untuk eksplorasi ini [23].

Sebagai penutup, penting untuk menggarisbawahi bahwa meskipun penelitian ini berhasil memberikan bukti empiris mengenai arsitektur fusi yang paling seimbang, model yang dihasilkan dalam kondisi saat ini belum siap untuk implementasi langsung di lingkungan dunia nyata yang tidak terkontrol. Keterbatasan utama terletak pada dataset yang digunakan, yang direkam dalam kondisi ideal dengan satu objek tunggal, pencahayaan konstan, dan latar belakang polos. Akibatnya, model yang ada saat ini kemungkinan besar akan mengalami kegagalan performa ketika dihadapkan pada tantangan-tantangan nyata seperti oklusi (objek terhalang sebagian), variasi pencahayaan yang drastis, latar belakang

yang kompleks, serta kebutuhan untuk mendeteksi beberapa objek sekaligus. Oleh karena itu, penelitian selanjutnya mutlak diperlukan untuk membangun di atas fondasi ini, dengan fokus pada peningkatan robustisitas model agar mampu mengatasi variabel-variabel dinamis tersebut sebelum dapat dianggap layak untuk aplikasi praktis.

DAFTAR PUSTAKA

- [1] Z. He, W. Feng, X. Zhao, and Y. Lv, "6D Pose Estimation of Objects: Recent Technologies and Challenges," *Appl. Sci.*, vol. 11, no. 1, p. 228, Dec. 2020, doi: 10.3390/app11010228.
- [2] B. Shan, D. Zhao, J. Yang, and H. Liu, "A Two-Stage Monocular Vision Detection Method for 6D Pose Estimation in Multi-Heterogeneous Robot Systems," *J. Phys. Conf. Ser.*, vol. 2832, no. 1, p. 012003, Aug. 2024, doi: 10.1088/1742-6596/2832/1/012003.
- [3] M. Abdelaal, R. M. A. Farag, M. S. Saad, A. Bahgat, H. M. Emara, and A. El-Dessouki, "Uncalibrated stereo vision with deep learning for 6-DOF pose estimation for a robot arm system," *Robot. Auton. Syst.*, vol. 145, p. 103847, Nov. 2021, doi: 10.1016/j.robot.2021.103847.
- [4] S. Alaba, A. Gurbuz, and J. Ball, "A Comprehensive Survey of Deep Learning Multisensor Fusion-based 3D Object Detection for Autonomous Driving: Methods, Challenges, Open Issues, and Future Directions," Aug. 23, 2022. doi: 10.36227/techrxiv.20443107.v2.
- [5] Rainhard and T. Mulyana, "PENENTUAN AREA WAJAH MENGGUNAKAN ALGORITMA VIOLA-JONES UNTUK APLIKASI MANIPULASI WARNA CITRA DIGITAL," Mar. 2025, doi: <http://dx.doi.org/10.30813/j-alu.v2i2.8152>.
- [6] Z. Xu, X. Yuan, T. Zhou, and L. Fang, "A multichannel optical computing architecture for advanced machine vision," *Light Sci. Appl.*, vol. 11, no. 1, p. 255, Aug. 2022, doi: 10.1038/s41377-022-00945-y.
- [7] J. Guan, Y. Hao, Q. Wu, S. Li, and Y. Fang, "A Survey of 6DoF Object Pose Estimation Methods for Different Application Scenarios," *Sensors*, vol. 24, no. 4, p. 1076, Feb. 2024, doi: 10.3390/s24041076.
- [8] C. Sahin, G. Garcia-Hernando, J. Sock, and T.-K. Kim, "A review on object pose recovery: From 3D bounding box detectors to full 6D pose estimators," *Image Vis. Comput.*, vol. 96, p. 103898, Apr. 2020, doi: 10.1016/j.imavis.2020.103898.
- [9] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the Continuity of Rotation Representations in Neural Networks," June 08, 2020, *arXiv*: arXiv:1812.07035. doi: 10.48550/arXiv.1812.07035.
- [10] M. Kalaitzakis, B. Cain, S. Carroll, A. Ambrosi, C. Whitehead, and N. Vitzilaios, "Fiducial Markers for Pose Estimation: Overview, Applications and Experimental Comparison of the ARTag, AprilTag, ArUco and STag Markers," *J. Intell. Robot. Syst.*, vol. 101, no. 4, p. 71, Apr. 2021, doi: 10.1007/s10846-020-01307-9.

- [11] G. Verma and R. Doriya, Eds., *Deep Learning: Theory, Architectures and Applications in Speech, Image and Language Processing*. BENTHAM SCIENCE PUBLISHERS, 2023. doi: 10.2174/97898150792101230101.
- [12] T. S. Jordan and S. Shridhar, “Usings CNNs to Estimate Depth from Stereo Imagery”.
- [13] N. An and W. Qi Yan, “Multitarget Tracking Using Siamese Neural Networks,” *ACM Trans. Multimed. Comput. Commun. Appl.*, vol. 17, no. 2s, pp. 1–16, June 2021, doi: 10.1145/3441656.
- [14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” Mar. 02, 2022, *arXiv, Facebook AI Research, UC Berkeley*: arXiv:2201.03545. doi: 10.48550/arXiv.2201.03545.
- [15] S. Woo *et al.*, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” Jan. 02, 2023, *arXiv*: arXiv:2301.00808. doi: 10.48550/arXiv.2301.00808.
- [16] T. Pöllabauer, A. Pramod, V. Knauthe, and M. Wahl, “FAST GDRNPP: Improving the Speed of State-of-the-Art 6D Object Pose Estimation,” Sept. 18, 2024, *arXiv*: arXiv:2409.12720. doi: 10.48550/arXiv.2409.12720.
- [17] Y. Su, J. Rambach, A. Pagani, and D. Stricker, “SynPo-Net—Accurate and Fast CNN-Based 6DoF Object Pose Estimation Using Synthetic Training,” *Sensors*, vol. 21, no. 1, p. 300, Jan. 2021, doi: 10.3390/s21010300.
- [18] K. C. Nugraha and E. M. Sipayung, “DETEKSI PLAT NOMOR KENDARAAN MENGGUNAKAN ALGORITMA YOLO,” no. 2, 2023, doi: <http://dx.doi.org/10.30813/j-alu.v6i2.4739>.
- [19] A. Navarro and M. Loaiza, “High-quality Voxel Reconstruction from Stereoscopic Images,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 3, 2022, doi: 10.14569/IJACSA.2022.0130376.
- [20] M. I. Pangestu, J. A. Ginting, I. G. N. Suryantara, and M. Marvelino, “OPTIMASI PERTANIAN DI BEKASI UTARA: PREDIKSI CURAH HUJAN DAN REKOMENDASI TANAMAN DENGAN MENGGUNAKAN MODEL REGRESI LINIER,” 2024, doi: <http://dx.doi.org/10.30813/j-alu.v7i2.8081>.
- [21] S. Garrido and A. Panov, “Detection of ArUco Markers.” [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
- [22] C. Yuanwei, M. Hairi Mohd Zaman, and M. Faisal Ibrahim, “A Review on Six Degrees of Freedom (6D) Pose Estimation for Robotic Applications,” *IEEE Access*, vol. 12, pp. 161002–161017, 2024, doi: 10.1109/ACCESS.2024.3487263.
- [23] K. Chen, S. James, C. Sui, Y.-H. Liu, P. Abbeel, and Q. Dou, “StereoPose: Category-Level 6D Transparent Object Pose Estimation from Stereo Images via